



OSTİM TEKNİK ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
Yazılım Mühendisliği Bölümü

ULUSLARARASI KONGRE TAM METNİ

VI. International British Congress on Interdisciplinary Scientific Research & Practices

Zaman Sayar: Bütünleşik Takvim, Görev, Not ve Sorumluluk Yönetimi Uygulamasının Tasarımı ve Gerçekleştirilmesi

Design and Implementation of an Integrated Calendar, Task, Note, and Responsibility Management Application: Zaman Sayar

Yazar	Samed Alp ARSLAN
Öğrenci No	220205012
ORCID	0009-0001-8341-5142
E-posta	220205012@ostimteknik.edu.tr
Kurum	OSTİM Teknik Üniversitesi, Yazılım Mühendisliği Bölümü
Akademik Danışman	Dr. Öğr. Üyesi Ramin ABBASZADI
Çalışma Türü	VI. International British Congress Tam Metin
Kabul Bilgisi	Ref: AC-06036 – Accepted for Oral Presentation and Proceedings
Kabul Tarihi	15/05/2026
Kongre Tarihi ve Yeri	18–21 Haziran 2026, Londra / Birleşik Krallık

London, United Kingdom, 2026

KONGRE BEYANI VE ETİK AÇIKLAMA

Bu metin, **Zaman Sayar** başlıklı çalışmanın VI. International British Congress on Interdisciplinary Scientific Research & Practices kapsamında **AC-06036** referans numarasıyla, **15/05/2026** tarihinde sözlü sunum ve kongre bildiri kitabına dahil edilmek üzere kabul edilen tam metin akademik ve teknik rapor sürümü olarak hazırlanmıştır. Kabul belgesinde yer alan İngilizce başlık “Design and Implementation of an Integrated Calendar, Task, Note, and Responsibility Management Application: Zaman Sayar” biçimindedir. Çalışma; zaman planlama, görev takibi, not alma, RACI temelli sorumluluk yönetimi, OCR destekli görev çıkarımı, GitHub entegrasyonu, ZamanChat ve makine öğrenmesi tabanlı akıllı öneri bileşenlerini tek bir web uygulaması içinde ele almaktadır.

Bu dosya, ders içi onay sayfası mantığından çok kongre tam metni düzenine uygun biçimde yapılandırılmıştır. Başlık sayfası, özetler, yöntem, bulgular, tartışma, sonuç, kaynaklar ve ekler; hem akademik sunum beklentilerini hem de teknik izlenebilirlik gereksinimini karşılayacak şekilde korunmuş ve güncellenmiştir.

Çalışmada gerçek kişisel veri kullanılmamış; makine öğrenmesi deneyleri sentetik ve anonim Türkçe görev veri kümesi üzerinde yürütülmüştür. Veri kümesinde kişi adları yerine rol etiketleri kullanılmış, değerlendirme sonuçları sabit tohumla tekrarlanabilir biçimde saklanmıştır. OCR, GitHub ve kullanıcı geri bildirimi gibi uygulama bileşenleri raporda gerçek çalışma biçimleriyle anlatılmış; kişisel erişim belirteci saklama, sentetik veri, YZ-destekli/benzetimli panel ve zayıf kümeleme sonucu gibi sınırlılıklar özellikle açıkça belirtilmiştir.

Bu raporda verilen ölçüm değerleri ve deneysel sonuçlar, proje kökünde tutulan geliştirme kanıtları ile makine-okur ölçüm çıktılarından türetilmiştir. `GELISTIRME_NOTLARI.md` dosyası iç geliştirme günlüğü ve kanıt belgesi niteliğindedir; rapor metnine doğrudan konmamış, ancak sayısal tutarlılığın izlenebilmesi için arka plan kayıt olarak korunmuştur.

TEŞEKKÜR

Bu çalışmanın ortaya çıkış ve gelişim sürecinde bilgi, yönlendirme ve akademik desteğini esirgemeyen danışmanım Dr. Öğr. Üyesi Ramin Abbaszadi'ye, projeye kattığı değer, gösterdiği titizlik ve çalışma sürecine sağladığı katkılar için içten teşekkürlerimi sunarım. Ayrıca projemin gelişmesinde önemli katkıları bulunan Dr. Cemil Baki Kıyak, Öğr. Gör. Aslı Göncü ve Dr. Çağla Aksoy'a destekleri için teşekkür ederim.

Mühendislik bakış açımın, teknik birikimimin ve mesleki vizyonumun gelişmesinde çok büyük emeği bulunan başta Eray Yağdereli, İclal Çetin Taş, Nooshin Nemati Tolakan ve Mustafa Sami Cücen olmak üzere hayatımda iz bırakan tüm kıymetli hocalarıma da en içten teşekkürlerimi sunarım.

Ayrıca, başta annem Ayfer Arslan, babam Şükrü Arslan ve ağabeyim Yaşar Arslan olmak üzere kıymetli aileme ve aile dostumuz Onur Sümer'e, her zaman yanımda oldukları ve desteklerini esirgemedikleri için teşekkür ederim.

Bu çalışma, yalnızca bir yazılım geliştirme sürecinin değil, aynı zamanda değerli rehberliklerle şekillenen uzun bir öğrenme yolculuğunun ürünüdür.

ÖZET

Bu çalışmada, zaman planlaması, görev takibi, not alma, rol temelli sorumluluk yönetimi, yazılım geliştirme izlenebilirliği ve bilgiye hızlı erişim gereksinimlerini tek bir web uygulaması altında birleştiren Zaman Sayar sistemi ele alınmıştır. Geliştirilen uygulama; takvim, görev ve alt görev yönetimi, ayrıntılı görev atama, sesli not, RACI matrisi, dinamik tablo, Excel/CSV veri aktarımı, istatistiksel gösterge ekranları, OCR destekli görselden görev çıkarımı, GitHub entegrasyonu, doğal dil ile salt-okunur veri sorgulama ve makine öğrenmesi tabanlı akıllı görev önerisi bileşenlerinden oluşmaktadır. Bu yönüyle çalışma, yalnızca bir arayüz prototipi değil; sunum katmanı, iş mantığı, veri erişimi ve yardımcı servisleri ayrıştırılmış bütünlük bir yazılım mühendisliği uygulamasıdır.

Sistem ASP.NET Core MVC, Razor, Entity Framework Core ve SQL Server temelli katmanlı bir mimari üzerinde geliştirilmiştir. OCR bileşeninde Python, Tesseract, EasyOCR ve Pillow kullanılarak görsel ön işleme, metin çıkarımı, etiket eşleme ve görev alanı üretimi gerçekleştirilmiştir. ZamanChat bileşeninde Node.js, Express ve Socket.IO ile çalışan, veritabanı şemasını okuyarak kullanıcı sorgularını sınırlı ve denetlenebilir biçimde yanıtlayan salt-okunur bir yardımcı arayüz tasarlanmıştır. GitHub entegrasyonu, kişisel erişim belirteci ile GitHub REST API üzerinden depo, konu, pull request, etkinlik ve takvim senkronu bilgilerini uygulama bağlamına taşımaktadır. Çalışmada bu mimari kararlar, veri modeli, modül tasarımları, arayüz çıktıları, algoritmik akışlar ve mühendislik kısıtları birlikte değerlendirilmiştir.

Bütünlük mimariye ek olarak sistem, sentetik ve anonim bir Türkçe görev veri kümesi üzerinde (1200 kayıt, 5-kat çapraz doğrulama) görev türü, kategorisi ve önceliğini sırasıyla 0,91 / 0,92 / 0,93 makro-F1 ile sınıflandıran; süreyi 1,20 saat MAE (MAPE %17,6; $R^2 = 0,90$) ile tahmin edip bütçe-eşleştirilmiş benzetimde gecikmeyi %44,3 azaltan; RACI sorumluluğunu %89,1 çapraz-doğrulama-dışı uyumla öneren ve Türkçe çıkarımsal özet üreten tekrarlanabilir bir makine öğrenmesi alt sistemi içermektedir. Özetleme bileşeninin okunabilirliği, N=20 YZ-destekli/benzetimli Likert panelinde 4,50/5 olarak raporlanmış; bu ölçüm gerçek insan denek çalışmasının yerine geçmediği için sınırlılıklar bölümünde ayrıca tartışılmıştır. Çalışma ayrıca 55 otomatik birim testiyle desteklenmiştir.

Değerlendirme sonucunda Zaman Sayar'ın dağınık dijital çalışma araçlarını tek bir operasyonel panelde birleştirdiği, görev-zaman ilişkisini görünür kıldığı, RACI ile sorumluluk paylaşımını izlenebilir hale getirdiği ve OCR/ZamanChat/GitHub/akıllı öneri gibi destek katmanlarıyla veri girişi, veri okuma ve karar destek yükünü azalttığı görülmüştür. Üretim ortamına geçiş için gerçek saha verisiyle yeniden doğrulama, uçtan uca test kapsamı, rol tabanlı yetkilendirme, hata izleme, erişilebilirlik ve kullanıcı deneyimi iyileştirmelerinin artırılması önerilmektedir.

Anahtar Kelimeler: ASP.NET Core MVC, görev yönetimi, takvim uygulaması, RACI, OCR, Entity Framework Core, SQL Server, ZamanChat, GitHub entegrasyonu, makine öğrenmesi, metin sınıflandırma, TF-IDF, TextRank özetleme, tekrarlanabilir değerlendirme

ABSTRACT

This study presents Zaman Sayar, a web-based system that integrates time planning, task tracking, note taking, role-based responsibility management, software-development traceability, and rapid access to information within a single application. The developed system consists of calendar management, task and subtask management, detailed task assignment, audio notes, RACI matrix support, dynamic tables, Excel/CSV data import, statistical dashboards, OCR-supported task extraction from images, GitHub integration, a read-only natural language querying component, and a machine-learning-based intelligent task suggestion module. In this respect, the project is not merely an interface prototype; it is an integrated software engineering application with separated presentation, business logic, data access, and auxiliary service layers.

The system was implemented on a layered architecture based on ASP.NET Core MVC, Razor, Entity Framework Core, and SQL Server. The OCR component uses Python, Tesseract, EasyOCR, and Pillow to perform image preprocessing, text extraction, label matching, and task field generation. The ZamanChat component is a read-only assistant interface based on Node.js, Express, and Socket.IO; it inspects the database schema and answers user queries within controlled boundaries. The GitHub integration retrieves repositories, issues, pull requests, activity, releases, and calendar-synchronizable due dates through the GitHub REST API by using a user/team-scoped personal access token. The report evaluates architectural decisions, data models, module designs, user interface outputs, algorithmic flows, and engineering constraints together.

Beyond integration, the system now includes a reproducible machine-learning subsystem that, on a synthetic and anonymized Turkish task corpus (1200 records, 5-fold cross-validation), classifies task type, category, and priority with macro-F1 scores of 0.91, 0.92, and 0.93 respectively; estimates task duration with a mean absolute error of 1.20 hours (MAPE 17.6%, $R^2 = 0.90$) and yields a 44.3% reduction in simulated lateness under a budget-matched scenario; suggests RACI responsibility with 89.1% out-of-fold agreement; and produces extractive Turkish summaries that attain a readability of 4.50/5 on a 20-rater AI-assisted/simulated Likert panel. The project is further supported by 55 automated unit tests. Since the readability panel is not a real human-subject study, it is reported transparently as an AI-assisted/simulated evaluation and treated as a limitation.

The evaluation shows that Zaman Sayar combines fragmented digital work tools into a single operational panel, makes the relationship between tasks and time visible, improves traceability of responsibility sharing through RACI, and reduces manual data entry, data retrieval, and decision-support effort through OCR, ZamanChat, GitHub, and intelligent suggestion support layers. Future improvements should focus on real-world validation, end-to-end test coverage, role-based authorization, error monitoring, accessibility, and user-experience maturity

before production-level use.

Keywords: ASP.NET Core MVC, task management, calendar application, RACI, OCR, Entity Framework Core, SQL Server, ZamanChat, GitHub integration, machine learning, text classification, TF-IDF, TextRank summarization, reproducible evaluation

İçindekiler

KONGRE BEYANI VE ETİK AÇIKLAMA	i
ÖZET	iii
ABSTRACT	v
İÇİNDEKİLER	vii
TABLolar LİSTESİ	xi
ŞEKİLLER LİSTESİ	xii
SİMGELER VE KISALTMALAR	xiii
0.1 KISALTMALAR	xiii
0.2 SİMGELER	xiii
1 GİRİŞ	1
1.1 Amaç	1
1.2 Giriş	1
1.3 Kapsam ve sınırlar	2
1.4 Çalışmanın özgün katkıları	2
1.5 Raporun organizasyonu	3
2 GENEL KAVRAMLAR	4
2.1 Görev yönetimi ve zaman planlama	4
2.2 Kişisel bilgi yönetimi	5
2.3 RACI yaklaşımı	5
2.4 OCR destekli bilgi çıkarımı	6
2.5 Web tabanlı yazılım mimarisi	7
2.6 Doğal dil arayüzü ve salt-okunur güvenlik sınırı	7
2.7 Veri modeli, yapılandırılmış veri ve izlenebilirlik	8

2.8	Makine öğrenmesi ile Türkçe görev metni işleme	9
3	KAYNAK ARAŞTIRMASI	11
3.1	Görev ve takvim odaklı sistemler	11
3.2	Kişisel bilgi yönetimi, hatırlatma ve bağlam sürekliliği	12
3.3	Sorumluluk yönetimi ve RACI kullanımı	12
3.4	Yapılandırılmış ve esnek veri yönetimi	13
3.5	OCR destekli bilgi çıkarımı	13
3.6	Doğal dil arayüzleri ve güvenli veri erişimi	14
3.7	Web tabanlı mimari, katmanlı tasarım ve sürdürülebilirlik	14
3.8	Görev metni sınıflandırma ve özetlemede makine öğrenmesi	15
3.9	Literatürden türetilen tasarım çıkarımları	15
3.10	Zaman Sayar'ın konumlandırılması	16
4	MATERYAL VE YÖNTEM	18
4.1	Geliştirme yaklaşımı	18
4.2	Sistem gereksinimleri	18
4.3	Yazılım mimarisi	19
4.4	Proje yapısı	20
4.5	Veri modeli	20
4.6	Modül tasarımı	21
4.7	OCR iş akışı	22
4.8	ZamanChat iş akışı ve güvenlik sınırı	23
4.9	GitHub Entegrasyonu	24
4.10	Makine öğrenmesi alt sistemi (zaman_ai)	25
4.11	Veri kümesi ve etik	26
4.12	Öznitelik çıkarımı ve modeller	26
4.13	Değerlendirme protokolü	27
4.14	Değerlendirme göstergeleri	27
5	ARAŞTIRMA SONUÇLARI VE TARTIŞMA	29

5.1	Uygulama çıktıları	29
5.2	Fonksiyonel doğrulama	29
5.3	A1: Görev türü, kategori ve öncelik sınıflandırması	30
5.4	A2: Süre tahmini ve gecikme azaltımı	31
5.5	A3: Not özetleme	32
5.6	A4: RACI sorumluluk önerisi	32
5.7	EK-3: Görev metni kümeleme	33
5.8	A5: Prototip, ölçüm raporu, ER şeması ve anonim veri	33
5.9	Akıllı Görev Önerisi Arayüzü (Prototip)	34
5.10	Arayüz sonuçları	35
5.11	Mühendislik açısından güçlü yönler	36
5.12	Sınırlılıklar	37
5.13	Tartışma	38
6	SONUÇLAR VE ÖNERİLER	39
6.1	Sonuçlar	39
6.2	Öneriler	39
	KAYNAKLAR	41
A	Ek-1: Teknik Bileşen Envanteri	44
B	Ek-2: Kaynak Kod Parçaları	45
B.1	Uygulama Başlatma ve Bağımlılıkların Kaydı	45
B.2	Veri Erişim Katmanı ve Entity İlişkileri	45
B.3	OCR Sonucundan Görev Kaydı Oluşturma	46
B.4	Web Uygulamasından Python OCR Betiğinin Çalıştırılması	47
B.5	Python Tarafında OCR Motoru Seçimi	48
B.6	ZamanChat İçin Salt-Okunur Socket.IO Akışı	48
B.7	Dinamik Şema Okuma Yaklaşımı	49
C	Ek-3: Veritabanı Şeması	50

Tablo Listesi

1	Çalışmanın mühendislik hedefleri	1
2	Temel kavramların projedeki karşılığı	10
3	Literatürden türetilen tasarım çıkarımları	16
4	Mevcut araç yaklaşımları ile Zaman Sayar'ın karşılaştırılması	17
5	Zaman Sayar sistem gereksinimleri	18
6	Mimaride kullanılan başlıca teknolojiler	19
7	Controller ve sorumluluk haritası	20
8	Modül bazlı tasarım özeti	22
9	ZamanChat karar akışı	23
10	zaman_ai modülleri ve araştırma hedefleri	26
11	Fonksiyonel doğrulama özeti	30
12	A1 – Sınıflandırma başarımı (1200 kayıt, 5-kat çapraz doğrulama)	31
13	A2 – Süre tahmini başarımı (5-kat çapraz doğrulama)	31
14	A2 – Bütçe-eşleştirilmiş gecikme azaltımı benzetimi	31
15	A3 – TextRank özet kalitesi (otomatik vekil okunabilirlik, 1–5)	32
16	A3 – Okunabilirlik Likert paneli (N=20, YZ-destekli/benzetimli; birincil ölçüt)	32
17	A4 – RACI önerisi başarımı ve kabul oranı (5-kat çapraz doğrulama)	33
18	EK-3 – Görev metni kümeleme metrikleri	33
19	TÜBİTAK 2209-A hedeflerine ulaşma özeti	35
20	Proje bileşenlerinin teknik özeti	44
21	Önerilen kalite güvence kontrol listesi	44
22	Veritabanı ilişki özeti	51
23	Başlıca veri varlıkları ve sorumlulukları	51

Şekil Listesi

1	Görev yönetimi ile zaman planlama arasındaki kavramsal ilişki	4
2	Kişisel bilgi yönetiminde bağlamın korunması	5
3	RACI rollerinin görev çevresindeki anlamı	6
4	OCR destekli görev çıkarımı kavramsal akışı	6
5	MVC tabanlı web mimarisinin Zaman Sayar'daki kavramsal karşılığı	7
6	ZamanChat için doğal dil arayüzü ve salt-okunur güvenlik sınırı	8
7	Yapılandırılmış veri modelinden izlenebilirliğe geçiş	8
8	Görev metninden öznitelik çıkarımı ve karar destek çıktısı üretimi	9
9	Zaman Sayar yazılım mimarisi	19
10	Zaman Sayar veri modeli özeti	21
11	OCR destekli görev çıkarımı iş akışı	23
12	Uygulama içi GitHub entegrasyonu: bağlanan hesabın profili, depo sayıları ve dil etiketleriyle birlikte depo kartları. Üst sekmeler depo, gösterge paneli, konular, pull request, etkinlik ve sürüm görünümüne erişim sağlar.	25
13	Zaman Sayar ana paneli, görev listesi ve ZamanChat görünümü	29
14	Akıllı Görev Önerisi (zaman_ai) arayüzü: serbest metinden tür/kategori-/öncelik (A1), tahmini süre/hatırlatma (A2), RACI sorumluluğu (A4) ve çıkarımsal özet (A3) üretimi; ayrıca insan-döngüde geri bildirim (kabul/ertele/ret), canlı kabul oranı ve geri bildirim inceleme/silme.	35
15	Takvim ve istatistik modülleri	36
16	RACI ve dinamik tablo arayüzleri	36
17	OCR ve ZamanChat destek ekranları	36
18	Zaman Sayar veritabanı şeması ve temel ilişki grupları	50

SİMGELER VE KISALTMALAR

0.1 KISALTMALAR

API	Application Programming Interface
ARI	Adjusted Rand Index (Düzeltilmiş Rand İndeksi)
ASP.NET	Active Server Pages .NET
CSV	Comma-Separated Values
CV	Cross-Validation (Çapraz Doğrulama)
EF Core	Entity Framework Core
F1	F1 Skoru (kesinlik–duyarlılık harmonik ortalaması)
HGB	HistGradientBoosting (Histogram Tabanlı Gradyan Artırma)
JSON	JavaScript Object Notation
LSA	Latent Semantic Analysis (Örtük Anlamsal Analiz)
MAE	Mean Absolute Error (Ortalama Mutlak Hata)
MAPE	Mean Absolute Percentage Error (Ortalama Mutlak Yüzde Hata)
ML	Machine Learning (Makine Öğrenmesi)
MVC	Model-View-Controller
NMI	Normalized Mutual Information (Normalize Karşılıklı Bilgi)
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
RACI	Responsible, Accountable, Consulted, Informed
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
SQL	Structured Query Language
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
UX	User Experience

0.2 SİMGELER

T	Tamamlanma oranı; tamamlanan görev sayısının toplam görev sayısına oranı.
$N_{tamamlanan}$	Tamamlanan görev sayısı.
N_{toplam}	Sistemde değerlendirilen toplam görev sayısı.
$Y(d)$	Belirli bir d günü için hesaplanan iş yoğunluğu değeri.
$w(p_i)$	i numaralı görevin önceliğine karşılık gelen ağırlık katsayısı.
c_i	i numaralı görevin tamamlanma durumunu gösteren ikili değişken.
S_m	OCR motoru m için hesaplanan seçim/başarı puanı.
F_m	OCR çıktısından çıkarılabilen görev alanı sayısı.
E_m	OCR çıktısındaki etiket eşleşme başarısı.
Q_m	OCR sonucunda elde edilen ham metnin okunabilirlik düzeyi.
C_m	OCR motorunun ürettiği güven değeri.
C_{RACI}	RACI rol kapsama oranı.

1 GİRİŞ

1.1 Amaç

Bu bitirme projesinin amacı, bireysel ve ekip temelli iş akışlarında kullanılabilecek, modüler fakat tek panelden yönetilebilen bir zaman ve görev yönetimi uygulaması tasarlamak ve gerçekleştirmektir. Uygulama, klasik bir takvim veya yapılacaklar listesi yaklaşımının ötesine geçerek aşağıdaki mühendislik hedeflerini karşılamayı amaçlamaktadır:

Tablo 1: Çalışmanın mühendislik hedefleri

Hedef	Açıklama
Bütünleşik iş akışı	Takvim, görev, not, tablo, RACI ve istatistik ekranlarını aynı sistem içinde erişilebilir hale getirmek.
Modüler yazılım mimarisi	Sunum, iş mantığı, veri erişimi ve yardımcı servisleri ayrıştırarak bakım yapılabilir bir yapı kurmak.
İzlenebilir veri modeli	Görev, takvim, not, yorum, tablo ve OCR kayıtlarını ilişkisel bir veritabanı omurgasında saklamak.
Yardımcı otomasyon	Görselden görev çıkarımı ve doğal dil ile veri sorgulama gibi destek katmanlarıyla manuel iş yükünü azaltmak.
Akıllı karar destek	Türkçe görev metninden tür, kategori, öncelik, süre, özet ve RACI sorumluluğu önerisi üretmek kullanıcı kararını desteklemek.
Akademik savunulabilirlik	Sistemi yalnızca ekranlardan oluşan bir uygulama olarak değil, mimari kararları, veri modeli ve sınırlılıklarıyla açıklanabilir bir mühendislik ürünü olarak sunmak.

1.2 Giriş

Dijital çalışma ortamlarında bireyler ve küçük ekipler çoğu zaman takvim, yapılacaklar listesi, not defteri, elektronik tablo, görev atama aracı ve raporlama ekranı gibi ayrı araçlar arasında geçiş yapmak zorunda kalmaktadır. Bu parçalı kullanım modeli ilk bakışta esnek görünse de zaman içinde veri tekrarına, bağlam kaybına, görev sahipliğinin belirsizleşmesine ve günlük iş yükünün bütüncül biçimde izlenememesine neden olmaktadır. Bir görevin tarihi bir takvimde, açıklaması başka bir notta, sorumlusu ayrı bir mesajlaşma kanalında, ölçülebilir durumu ise bağımsız bir tabloda kaldığında, yazılım desteği iş akışını hızlandırmak yerine kullanıcının zihinsel yükünü artırabilmektedir.

Zaman Sayar projesi bu probleme uygulamaya dönük bir çözüm üretmek amacıyla geliştirilmiştir. Proje, zaman planlaması ve görev yönetimini merkezde tutan; not, ses kaydı, RACI temelli sorumluluk paylaşımı, dinamik tablo, Excel/CSV aktarımı, istatistiksel izleme, OCR ile görselden görev çıkarımı, GitHub entegrasyonu, doğal dil ile salt-okunur sorgulama ve makine öğrenmesi tabanlı akıllı görev önerisi işlevlerini aynı web uygulaması içinde top-

layan bütünsel bir sistemdir. Çalışmanın temel motivasyonu, kullanıcının gün içindeki işi yalnızca kaydetmesini değil, bu işi zaman, sorumluluk, açıklama, yazılım geliştirme izlenebilirliği ve veri bağlamı ile birlikte yönetebilmesini sağlamaktır.

1.3 Kapsam ve sınırlar

Çalışmanın kapsamı, ASP.NET Core MVC tabanlı bir web uygulamasının tasarımı, uygulanması ve mühendislik açısından değerlendirilmesini içermektedir. Uygulamada takvim, görev ve alt görev yönetimi, ayrıntılı görev atama, not ve sesli not, RACI matrisi, dinamik tablo, hazır tablo yükleme, istatistik ekranları, OCR destekli görev çıkarımı, GitHub entegrasyonu, ZamanChat adlı salt-okunur sohbet yardımcısı ve zaman_ai adlı makine öğrenmesi alt sistemi yer almaktadır. Bu modüller aynı veritabanı ve aynı web uygulaması omurgası üzerinde çalışacak biçimde tasarlanmıştır.

Proje bir bitirme projesi ve kongre tam metni kapsamında değerlendirildiği için bazı üretim seviyesi konular geliştirmeye açık bırakılmıştır. Örneğin kapsamlı rol tabanlı yetkilendirme, uçtan uca test paketi, merkezi loglama, dağıtık servis izleme ve çok kullanıcıli üretim ölçekleme bu çalışmanın ana teslim hedefi değildir. Bununla birlikte makine öğrenmesi alt sistemi için 55 birim testi hazırlanmış; mevcut sistem, daha geniş test ve üretim güvenliği çalışmalarının ileride eklenebilmesine olanak sağlayan modüler bir temel sunmuştur.

1.4 Çalışmanın özgün katkıları

Bu çalışmanın özgün katkısı, tek tek bilinen modülleri basitçe yan yana getirmesinden değil; bu modülleri zaman, görev ve sorumluluk yönetimi problemi etrafında birlikte çalışacak biçimde tasarlamasından kaynaklanmaktadır. Proje kapsamında öne çıkan katkılar aşağıdaki gibidir:

1. Takvim, görev, not, RACI, dinamik tablo, dosya aktarımı, istatistik, OCR ve sohbet bileşenlerini aynı ürün mantığı içinde birleştiren bütünsel bir çalışma paneli geliştirilmiştir.
2. ASP.NET Core MVC, Entity Framework Core ve SQL Server üzerinde katmanlı bir yazılım mimarisi kurulmuş; Python ve Node.js tabanlı yardımcı servisler çekirdek sisteme kontrollü biçimde bağlanmıştır.
3. Tesseract ve EasyOCR çıktısını karşılaştıran, Türkçe etiket eş anlamlılarını kullanan ve sonuçları görev alanlarına dönüştüren OCR destekli görev çıkarımı tasarlanmıştır.
4. ZamanChat bileşeni ile veritabanı şemasını okuyabilen, yalnızca SELECT temelli sorgulama yapan ve yazma amaçlı niyetleri reddeden denetlenebilir bir doğal dil arayüzü oluşturulmuştur.

5. Uygulamanın arayüzleri, veri modeli, algoritmik kararları ve sınırlılıkları mühendislik raporu formatında şekil, tablo ve formüllerle açıklanmıştır.
6. Türkçe görev metinlerinden tür, kategori ve öncelik çıkaran; süre ve hatırlatma öneren; RACI sorumluluğu ve çıkarımsal özet üreten, tekrarlanabilir ve sabit tohumlu bir makine öğrenmesi alt sistemi (zaman_ai) tasarlanmış ve mevcut mimariye süreç tabanlı bir servis çağrısıyla entegre edilmiştir.
7. Makine öğrenmesi hedefleri sentetik ve anonim veri üzerinde 5-kat çapraz doğrulama ile ölçülmüş; sınıflandırma, regresyon, özetleme, RACI önerisi ve kümeleme sonuçları tablo ve güven aralığı bilgileriyle raporlanmıştır.
8. Önceki sürümde bulunan test boşluğu kısmen kapatılmış; zaman_ai alt sistemi için 55 otomatik birim testi hazırlanmıştır.

1.5 Raporun organizasyonu

Raporun ikinci bölümünde görev yönetimi, kişisel bilgi yönetimi, RACI, OCR, makine öğrenmesi ve web tabanlı sistem mimarisiyle ilgili temel kavramlar açıklanmaktadır. Üçüncü bölümde ilgili çalışmalar ve mevcut araç yaklaşımları değerlendirilmekte, Zaman Sayar'ın bu yaklaşımlar içindeki konumu tartışılmaktadır. Dördüncü bölümde sistem gereksinimleri, yazılım mimarisi, veri modeli, modül tasarımı, OCR, ZamanChat, GitHub entegrasyonu ve zaman_ai akışları ayrıntılı biçimde sunulmaktadır. Beşinci bölümde uygulama çıktıları, arayüz görselleri, fonksiyonel değerlendirme, makine öğrenmesi sonuçları ve sınırlılıklar ele alınmaktadır. Altıncı bölümde ise sonuçlar ve gelecek geliştirme önerileri verilmektedir.

2 GENEL KAVRAMLAR

Bu bölüm, raporun sonraki kısımlarında kullanılan temel kavramları açıklamak amacıyla hazırlanmıştır. Zaman Sayar; görev yönetimi, takvim, not, RACI, dinamik tablo, OCR, doğal dil ile sorgulama ve web tabanlı yazılım mimarisi gibi farklı kavramları aynı sistem içinde birleştirdiği için, bu kavramların raporun başında açık biçimde tanımlanması önemlidir. Böylece projeyi daha önce incelememiş bir okuyucu, uygulamanın yalnızca ekranlardan oluşmadığını; zaman, görev, bilgi, sorumluluk ve veri yönetimi kavramlarını birlikte ele alan bir yazılım mühendisliği çalışması olduğunu daha kolay anlayabilir.

2.1 Görev yönetimi ve zaman planlama

Görev yönetimi, yalnızca yapılacak işlerin listelenmesi anlamına gelmemektedir. Etkili bir görev yönetim sisteminde işin ne olduğu, kime ait olduğu, hangi tarihe bağlı olduğu, hangi önceliğe sahip olduğu ve tamamlanma durumunun nasıl izleneceği birlikte değerlendirilmelidir. Bellotti ve arkadaşları, görev yöneticilerinin kullanıcıya eyleme geçilebilir bilgi sunması gerektiğini vurgulamaktadır [1]. Bu bakış açısı, görev kaydını pasif bir liste olmaktan çıkararak karar destekleyici bir yapıya dönüştürmektedir.

Bu bağlamda *görev*, yapılması gereken iş birimini; *alt görev*, daha büyük bir işin daha küçük ve izlenebilir parçalara ayrılmış halini; *görev atama* ise işin belirli kişi, tarih, öncelik ve açıklama bilgileriyle sorumlulaştırılmasını ifade eder. Zaman Sayar'da görevler yalnızca metin olarak saklanmaz; durum, öncelik, teslim tarihi, atanan kişi, açıklama, ek dosya ve takip alanlarıyla birlikte yönetilir. Böylece sistem, basit bir yapılacaklar listesi olmaktan çıkarak işin bağlamını da koruyan bir görev yönetimi aracına dönüşür.

Takvim kullanımı da benzer biçimde yalnızca randevu saklama işleviyle sınırlı değildir. Takvim; yaklaşan işlerin görülmesi, yoğun günlerin fark edilmesi, hazırlık süresinin planlanması ve iş yükünün zamana yayılması için kullanılan temel bir bilgi düzenleme aracıdır [2, 3]. Bu nedenle zaman planlama ile görev yönetimi arasında doğal bir bağ bulunmaktadır. Zaman Sayar, bu iki kavramı aynı uygulama omurgasında birleştirerek kullanıcının işi hem liste hem de zaman eksenini üzerinde izlemesini amaçlamaktadır.



Şekil 1: Görev yönetimi ile zaman planlama arasındaki kavramsal ilişki

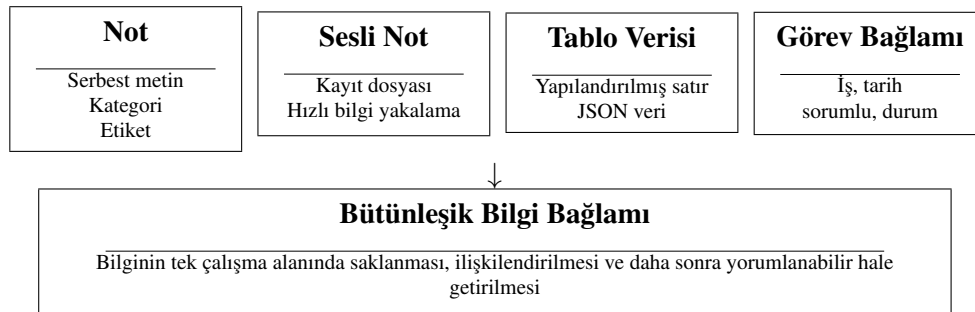
Şekil 1, görev yönetimi ile zaman planlamanın neden birlikte ele alınması gerektiğini özetlemektedir. Bir iş kaydı yalnızca açıklamadan oluştuğunda kullanıcı ne yapılacağını bilir;

ancak ne zaman yapılacağını, kimin sorumlu olduğunu ve işin hangi durumda bulunduğunu izlemekte zorlanabilir. Bu nedenle görev ve zaman bilgisinin aynı veri bağlamında tutulması, uygulamanın temel tasarım kararlarından biridir.

2.2 Kişisel bilgi yönetimi

Kişisel bilgi yönetimi, bireyin günlük çalışma sürecinde ürettiği veya kullandığı bilgileri daha sonra bulunabilir, anlaşılabilir ve kullanılabilir biçimde düzenlemesini ifade eder. Bu bilgi; kısa bir not, bir ses kaydı, bir görev açıklaması, bir tablo satırı, bir dosya eki, bir yorum veya bir takvim girdisi olabilir. Kişisel bilgi yönetimi literatürü, bilginin yalnızca saklanması yeterli olmadığını; daha sonra bulunabilir, yorumlanabilir ve eyleme dönüştürülebilir olması gerektiğini ortaya koymaktadır [4].

Bir göreve ilişkin notun, ses kaydının, ek açıklamanın, tablo satırının veya sorumluluk bilgisinin farklı sistemlerde kalması, bağlamın korunmasını zorlaştırmaktadır. Örneğin bir toplantıdan çıkan iş maddesi not uygulamasında, tarihi takvimde, sorumlusu mesajlaşma uygulamasında ve ilerleme durumu ayrı bir tabloda tutulduğunda, kullanıcı aynı işi anlamak için birden fazla yere bakmak zorunda kalır. Bu nedenle Zaman Sayar’da notlar, ses kayıtları, görevler, takvim kayıtları ve tablolar aynı çalışma alanı içinde ele alınmıştır.



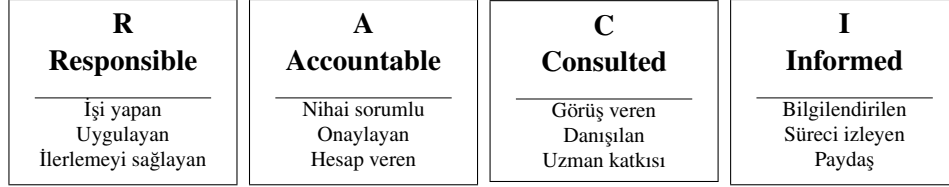
Şekil 2: Kişisel bilgi yönetiminde bağlamın korunması

Şekil 2, Zaman Sayar’ın not, sesli not, tablo ve görev bilgilerini aynı bağlamda değerlendirmesinin gerekçesini göstermektedir. Bu yaklaşım, kullanıcının bilgiyi yalnızca kaydetmesine değil, daha sonra aynı iş akışı içinde yeniden kullanılmasına da olanak tanır.

2.3 RACI yaklaşımı

RACI, ekip içi görev dağılımında rollerin açık biçimde tanımlanmasını sağlayan bir sorumluluk atama yaklaşımıdır. RACI kısaltması *Responsible*, *Accountable*, *Consulted* ve *Informed* rollerini ifade eder. Bu yapı, bir işin doğrudan sorumlusunu, nihai hesap verebilir kişiyi, görüşü alınacak kişileri ve bilgilendirilecek tarafları ayrıştırır [5]. Özellikle bir görev birden fazla kişiyi etkilediğinde, yalnızca “atanan kişi” bilgisini tutmak yeterli olmayabilir.

RACI modelinde *Responsible* rolü işi fiilen yapan kişiyi veya kişileri; *Accountable* rolü nihai karar ve sonuçtan sorumlu kişiyi; *Consulted* rolü görüşü alınması gereken paydaşları; *Informed* rolü ise süreç hakkında bilgilendirilecek kişileri temsil eder. Zaman Sayar’da RACI yapısının yer alması, görevin yalnızca bir kişiye atanmasını değil, görev çevresindeki iletişim ve sorumluluk ilişkilerinin de görünür hale gelmesini sağlamaktadır.



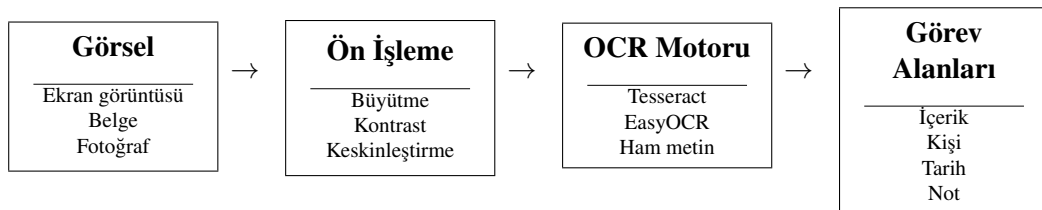
Şekil 3: RACI rollerinin görev çevresindeki anlamı

Şekil 3, RACI rollerinin birbirinden neden ayrılması gerektiğini görsel olarak özetlemektedir. Bu ayrım, özellikle ekip içi çalışma, görev devri, izleme ve raporlama süreçlerinde karışıklığı azaltır. Zaman Sayar’daki RACI modülü de bu nedenle yalnızca bir tablo görünümü değil, sorumluluk bilgisini görev yönetimiyle ilişkilendiren bir yönetim bileşeni olarak tasarlanmıştır.

2.4 OCR destekli bilgi çıkarımı

OCR, görsel veya belge üzerinde yer alan metnin makine tarafından okunabilir hale getirilmesini sağlayan bir teknolojidir. Tesseract, belge işleme alanında uzun süredir kullanılan açık kaynaklı OCR motorlarından biridir [6, 7]. EasyOCR ise çok dilli kullanım kolaylığı ve Python ekosistemiyle hızlı bütünleştirme olanağı sunmaktadır [8]. Bununla birlikte OCR çıktısı çoğu zaman yalnızca ham metin üretir; bu ham metnin uygulamada kullanılabilir hale gelmesi için ek bir yorumlama katmanına ihtiyaç vardır.

Zaman Sayar açısından OCR’nin değeri, görselden yalnızca metin okumak değildir. Asıl hedef; görev içeriği, atanan kişi, atayan kişi, son tarih ve not gibi alanları görselden çıkararak yapılandırılmış görev kaydına dönüştürmektir. Bu nedenle bu çalışmada OCR, bir “metin okuma” özelliği olarak değil, görselden yapılandırılmış görev üretimine yardım eden bir bileşen olarak ele alınmıştır. OCR sonucunun güven değeri ve ham metin bilgisi ayrıca saklanarak sistemin denetlenebilirliği de artırılmıştır.



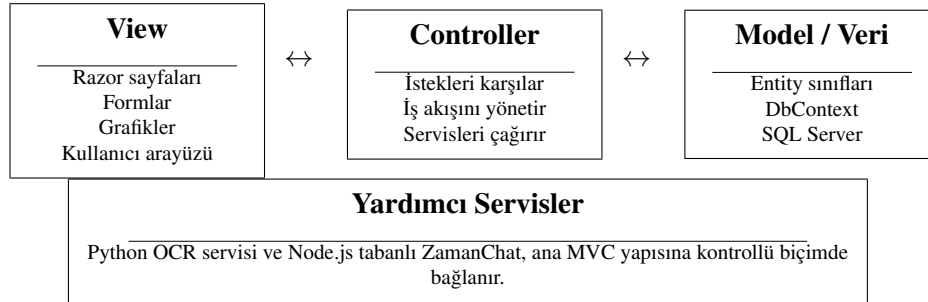
Şekil 4: OCR destekli görev çıkarımı kavramsal akışı

Şekil 4, OCR'nin raporda neden ayrı bir kavram olarak ele alındığını göstermektedir. Kullanıcı açısından bu işlem, görsel yükleyerek görev oluşturma kolaylığı sağlar. Yazılım mühendisliği açısından ise görsel doğrulama, ön işleme, metin çıkarımı, alan eşleme, güven puanı ve veritabanı kaydı gibi birden fazla teknik adım içerir.

2.5 Web tabanlı yazılım mimarisi

Web tabanlı uygulamalarda sürdürülebilirlik için sunum katmanı, iş mantığı ve veri erişimi arasında açık bir ayrım kurulması önemlidir. ASP.NET Core MVC bu ayrımı Model, View ve Controller yapısıyla desteklemekte; Entity Framework Core ise nesne yönelimli modeller ile ilişkisel veritabanı arasındaki eşlemeyi yönetmektedir [9, 10]. Zaman Sayar'da bu yaklaşım, modüllerin tek bir dosya veya tek bir ekran içinde birikmesini engellemek ve yazılımın ileride genişletilebilmesini kolaylaştırmak amacıyla kullanılmıştır.

Bu mimaride *Model*, uygulamada kullanılan veri varlıklarını; *View*, kullanıcının gördüğü arayüzleri; *Controller* ise kullanıcı isteğini alıp uygun iş mantığını çalıştıran katmanı temsil eder. Entity Framework Core gibi ORM araçları, bu modellerin SQL Server gibi ilişkisel veritabanlarında saklanmasını kolaylaştırır. Böylece geliştirici, veritabanı tabloları ile C# sınıfları arasındaki ilişkiyi daha düzenli ve sürdürülebilir biçimde yönetebilir.



Şekil 5: MVC tabanlı web mimarisinin Zaman Sayar'daki kavramsal karşılığı

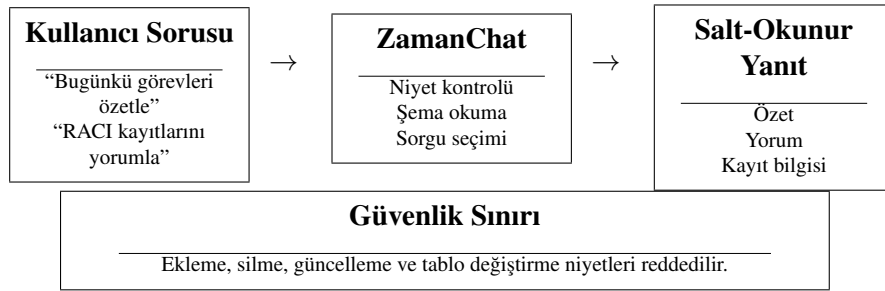
Şekil 5, Zaman Sayar'ın temel web mimarisini kavramsal olarak göstermektedir. Bu ayrım, hem raporun ilerleyen bölümlerinde açıklanan controller haritasını anlamayı kolaylaştırır hem de OCR ve ZamanChat gibi yardımcı bileşenlerin ana sisteme nasıl bağlandığını daha görünür hale getirir.

2.6 Doğal dil arayüzü ve salt-okunur güvenlik sınırı

Doğal dil arayüzü, kullanıcının sisteme belirli bir komut dili veya karmaşık filtre ekranı kullanmadan, günlük dile yakın ifadelerle soru sorabilmesini ifade eder. Zaman Sayar'daki ZamanChat bileşeni bu amaçla geliştirilmiş yardımcı bir katmandır. Kullanıcı belirli bir modüldeki kayıtları özetleme, yorumlama veya ayrıntılarını isteme gibi sorgular gönderebilir.

Ancak bu tür arayüzlerde en önemli konulardan biri güvenlik sınırının açık biçimde tanımlanmasıdır.

Salt-okunur tasarım, sistemin veritabanındaki bilgileri okuyabilmesini ancak veri ekleme, silme veya güncelleme gibi işlemleri gerçekleştirmemesini ifade eder. Bu yaklaşım, doğal dil arayüzünün kolaylık sağlamasını ama veri bütünlüğünü riske atmamasını hedefler. ZamanChat bu nedenle yazma niyeti taşıyan kullanıcı isteklerini reddedecek şekilde tasarlanmıştır. Böylece doğal dil ile sorgulama özelliği, ana uygulama mantığını riske atmadan sisteme eklenmiştir.

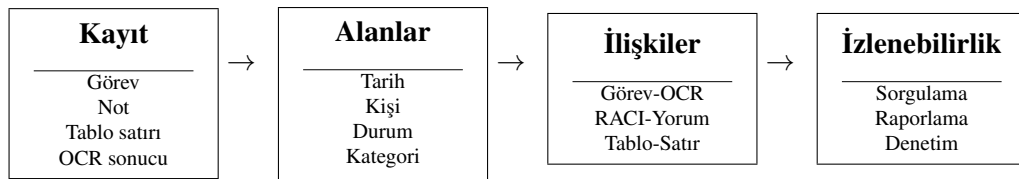


Şekil 6: ZamanChat için doğal dil arayüzü ve salt-okunur güvenlik sınırı

2.7 Veri modeli, yapılandırılmış veri ve izlenebilirlik

Veri modeli, uygulamada hangi bilgilerin hangi varlıklar altında tutulacağını ve bu varlıkların birbirleriyle nasıl ilişkileneceğini tanımlar. Bir yazılım sisteminin sürdürülebilirliği için ekranların güzel görünmesi tek başına yeterli değildir; ekranların arkasındaki veri yapısının da tutarlı, izlenebilir ve genişletilebilir olması gerekir. Zaman Sayar’da görev, takvim, not, RACI, dinamik tablo, hazır tablo ve OCR günlükleri ayrı varlık grupları olarak modellenmiştir.

Yapılandırılmış veri, belirli alanlara ayrılmış ve daha sonra sorgulanabilir hale getirilmiş veridir. Örneğin bir görev kaydında görev içeriği, atanan kişi, tarih ve durum alanlarının ayrı tutulması; bu bilginin takvimde gösterilmesini, istatistiklerde kullanılmasını ve ZamanChat tarafından okunmasını kolaylaştırır. İzlenebilirlik ise bir kaydın nasıl oluştuğunu, hangi bilgiye dayandığını ve hangi başka kayıtlarla ilişkili olduğunu takip edebilme yeteneğidir. OCR sonucunda ham metin, motor bilgisi ve güven değerinin saklanması bu nedenle önemlidir.

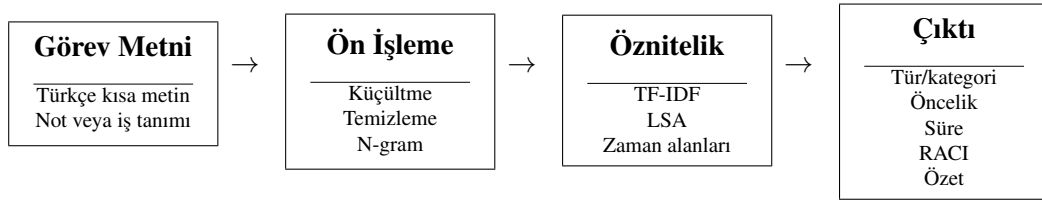


Şekil 7: Yapılandırılmış veri modelinden izlenebilirliğe geçiş

2.8 Makine öğrenmesi ile Türkçe görev metni işleme

Görev metinleri genellikle kısa, düzensiz ve bağlama bağımlı ifadelerdir. Türkçe gibi eklemeli dillerde aynı kökten türeyen biçimlerin farklı yüzey biçimleriyle görünmesi, bu metinlerin otomatik etiketlenmesini daha dikkatli bir ön işleme problemine dönüştürür. Bu tür metinlerin sınıflandırılmasında yaygın yaklaşım, metni sayısal özniteliklere dönüştürmek ve bu öznitelikler üzerinden denetimli bir model eğitmektir. TF-IDF yöntemi, terimlerin belge içindeki sıklığını ve tüm veri kümesi içindeki ayırt ediciliğini birlikte değerlendirerek kısa metinler için güçlü bir taban temsil sunar [24].

Bu çalışmada görev metinleri 1–2 sözcüklük n-gramlar üzerinden TF-IDF ile ağırlıklandırılmış; anlamsal yakınlığı daha düşük boyutlu bir uzayda temsil etmek için Örtük Anlamsal Analiz (LSA) yaklaşımı kullanılmıştır [25]. Sınıflandırma için lojistik regresyon ve histogram tabanlı gradyan artırma modelleri, basit bir kural temeliyle karşılaştırılmıştır. Süre tahmini regresyon problemi olarak, RACI rol önerisi ise çok sınıflı sınıflandırma problemi olarak ele alınmıştır. Not özetlemede ise çizge tabanlı TextRank yöntemi kullanılmıştır [26].



Şekil 8: Görev metninden öznitelik çıkarımı ve karar destek çıktısı üretimi

Şekil 8, makine öğrenmesi alt sisteminin raporda neden ayrı bir kavram olarak ele alındığını göstermektedir. Sistem, kullanıcı metnini yalnızca saklamaz; metni sınıflandırma, süre tahmini, sorumluluk önerisi ve özetleme için ölçülebilir bir karar destek girdisine dönüştürür.

Tablo 2: Temel kavramların projedeki karşılığı

Kavram	Genel anlamı	Zaman Sayar'daki karşılığı
Görev yönetimi	İşlerin durum, öncelik ve sahiplik bilgisiyle izlenmesi	Görev, alt görev ve görev atama modülleri
Zaman planlama	İşlerin tarih ve saat ekseninde düzenlenmesi	Gün, hafta ve ay görünümüne sahip takvim modülü
Kişisel bilgi yönetimi	Bilginin bağlamıyla birlikte saklanması	Not, sesli not, açıklama ve tablo kayıtları
RACI	Rol ve sorumlulukların ayrıştırılması	RACI matrisi ve rol bazlı görünüm ekranları
OCR	Görselden metin çıkarımı	Tesseract/EasyOCR destekli görev alanı üretimi
ORM	Nesne yönelimli modellerin ilişkisel veritabanıyla eşlenmesi	Entity Framework Core ve ApplicationDbContext yapısı
JSON	Esnek ve alan bazlı veri gösterimi	Dinamik tablo ve hazır tablo satır verileri
Doğal dil arayüzü	Kullanıcının metinle veri sorgulaması	ZamanChat salt-okunur sorgulama bileşeni
Makine öğrenmesi	Veriden örüntü öğrenerek karar desteği üretme	zaman_ai ile görev sınıflandırma, süre tahmini, RACI önerisi ve özetleme
İzlenebilirlik	Bir kaydın kaynağının ve ilişkilerinin takip edilebilmesi	OCR logları, RACI yorumları ve ilişkili görev kayıtları

3 KAYNAK ARAŞTIRMASI

Bu bölümde Zaman Sayar projesinin dayandığı akademik ve teknik kaynaklar, doğrudan proje bileşenleriyle ilişkilendirilerek ele alınmıştır. Proje yalnızca bir görev listesi veya yalnızca bir takvim uygulaması değildir; görev yönetimi, zaman planlama, kişisel bilgi yönetimi, sorumluluk matrisi, esnek veri girişi, OCR destekli bilgi çıkarımı, doğal dil ile veri okuma ve web tabanlı yazılım mimarisi gibi birden fazla alanın birlikte kullanıldığı bütünlüklü bir yazılım sistemidir. Bu nedenle kaynak araştırması, tek bir literatür başlığı altında değil, sistemin temel modüllerine karşılık gelen araştırma alanları üzerinden yapılandırılmıştır.

Literatür incelemesinin temel amacı, Zaman Sayar’da geliştirilen modüllerin hangi gereksinimlerden doğduğunu, mevcut yaklaşımların hangi yönlerden yeterli olduğunu ve projenin bu yaklaşımlar üzerine hangi ek değeri koyduğunu açıklamaktır. Böylece kaynak araştırması bölümü, yalnızca atıf yapılan bir metin olmaktan çıkarak projenin mühendislik kararlarını gerekçelendiren bir arka plan sunmaktadır.

3.1 Görev ve takvim odaklı sistemler

Görev yönetimi ve takvim sistemleri üzerine yapılan çalışmalar, kullanıcıların dijital araçlardan yalnızca kayıt tutma değil, işlerini eyleme dönüştürebilecekleri düzenli bir çalışma zemini beklediğini göstermektedir. Bellotti ve arkadaşları, kişisel görev yönetiminde kullanıcıların yapılacak işleri yalnızca listelemek istemediğini; bu işleri bağlam, öncelik, zaman ve sonraki eylem açısından takip etmek istediğini belirtmektedir [1]. Bu bakış açısı, görev yönetimini basit bir kontrol listesinden çıkararak karar destekleyici bir kullanıcı deneyimine dönüştürmektedir.

Takvim kullanımı da benzer biçimde yalnızca randevu saklama işleviyle sınırlı değildir. Payne, takvimlerin günlük işlerin zihinsel olarak organize edilmesinde önemli bir dış bellek aracı olduğunu vurgulamaktadır [2]. Tungare ve Perez-Quinones tarafından yapılan çalışma ise kişisel takvim kullanımının farklı kullanıcı alışkanlıklarına göre değiştiğini, bu nedenle takvim sistemlerinin esnek ve yorumlanabilir olması gerektiğini göstermektedir [3]. Bu çalışmalar birlikte değerlendirildiğinde, görev ve zaman bilgisinin ayrı ekranlarda kopuk biçimde tutulması yerine aynı veri bağlamı içinde ele alınması gerektiği görülmektedir.

Zaman Sayar bu literatürden hareketle görev ve takvim modüllerini birbirini tamamlayan iki ayrı görünüm olarak ele almaktadır. Kullanıcı bir görevi içerik, öncelik, sorumlu kişi, son tarih, açıklama ve tamamlanma durumu ile birlikte kaydedebilmekte; takvim ekranı ise aynı kayıtların zaman eksenindeki dağılımını görünür hale getirmektedir. Bu yaklaşım, görevlerin yalnızca oluşturulmasını değil, zaman içinde izlenmesini de desteklemektedir.

3.2 Kişisel bilgi yönetimi, hatırlatma ve bağlam sürekliliği

Kişisel bilgi yönetimi literatüründe temel problem, kullanıcının farklı zamanlarda ürettiği bilgi parçalarının daha sonra bulunabilir, anlaşılabilir ve kullanılabilir kalmasıdır. Jones, kişisel bilgi yönetimini bireyin bilgiyi toplama, düzenleme, saklama, bulma ve yeniden kullanma süreçleriyle açıklamakta; bilginin yalnızca saklanması yeterli olmadığını, ileride işlevsel biçimde kullanılabilmesi gerektiğini vurgulamaktadır [4]. Bu yaklaşım, not alma, görev açıklama, dosya tutma ve tablo verisi saklama gibi eylemlerin aynı çalışma bağlamında ele alınmasını gerektirmektedir.

Barreau ve Nardi'nin dosya düzenleme üzerine çalışması, kullanıcıların bilgiyi yalnızca arşivlemek için değil, aynı zamanda kendilerine hatırlatıcı olacak biçimde konumlandıklarını göstermektedir [17]. Whittaker ve Sidner ise e-posta kullanımında ortaya çıkan bilgi yükü problemini inceleyerek, iletişim araçlarının zamanla görev takibi ve kişisel bilgi yönetimi amacıyla da kullanılmaya başlandığını ortaya koymaktadır [18]. Bu bulgular, kullanıcıların tek amaçlı araçları çoğu zaman kendi çalışma ihtiyaçlarına göre dönüştürdüğünü göstermektedir.

Zaman Sayar'ın not ve sesli not modülleri bu bağlamda yalnızca serbest metin saklayan yardımcı ekranlar değildir. Notlar kategori, etiket ve ses kaydı gibi bağlam bilgileriyle birlikte tutulmakta; görev, takvim ve tablo modülleriyle aynı uygulama çatısı altında yer almaktadır. Böylece kullanıcı, bir işin yalnızca ne zaman yapılacağını değil, o işle ilgili açıklama, hatırlatma ve destekleyici bilgileri de aynı sistem içinde koruyabilmektedir.

3.3 Sorumluluk yönetimi ve RACI kullanımı

Ekip çalışmalarında bir görevin kim tarafından yapılacağı kadar, kararın kim tarafından onaylanacağı, kime danışılacağı ve kimin bilgilendirileceği de önemlidir. RACI yaklaşımı, görev çevresindeki rol ve sorumlulukları ayırtmak için kullanılan pratik bir sorumluluk atama çerçevesidir [5]. Bu çerçevede *Responsible* işi yapan kişiyi, *Accountable* nihai sorumluluğu taşıyan kişiyi, *Consulted* görüşü alınan tarafları, *Informed* ise süreçten haberdar edilmesi gereken paydaşları ifade eder.

Geleneksel görev yönetimi araçlarında sorumluluk bilgisi çoğu zaman yalnızca “atanan kişi” alanıyla temsil edilmektedir. Ancak gerçek çalışma süreçlerinde bir görevin yürütülmesi, onaylanması, danışılması ve bilgilendirilmesi gereken kişiler farklı olabilir. Bu ayrım yapılmadığında, görev tamamlanmış görünse bile iletişim ve karar sorumluluğu belirsiz kalabilir.

Zaman Sayar'da RACI modülünün ayrı bir bileşen olarak yer alması bu nedenle önemlidir. Sistem, bir görevi yalnızca yapılacak iş olarak değil, çevresinde rol dağılımı, ilerleme yüzdesi ve yorum geçmişi olan yönetilebilir bir kayıt olarak ele almaktadır. Bu yaklaşım, projenin bireysel görev takibinin ötesine geçerek ekip içi sorumluluk görünürlüğü sağlamasına

katkı vermektedir.

3.4 Yapılandırılmış ve esnek veri yönetimi

Bir yazılım sisteminin sürdürülebilirliği, yalnızca arayüz ekranlarının çalışmasına değil, arka plandaki veri modelinin tutarlı tasarlanmasına da bağlıdır. Chen'in varlık-ilişki modeli, gerçek dünyadaki nesnelerin ve aralarındaki ilişkilerin veritabanı tasarımında açık biçimde temsil edilebilmesi için temel bir yaklaşım sunmaktadır [19]. Bu yaklaşım, görev, kullanıcı, not, yorum, OCR çıktısı ve tablo satırı gibi farklı kavramların ayrı varlıklar olarak tanımlanmasını ve aralarındaki ilişkilerin izlenebilir hale gelmesini desteklemektedir.

Zaman Sayar'ın veri modeli bu açıdan iki yönlü bir gereksinimi karşılamaktadır. Birinci gereksinim, görev, not, RACI ve OCR kayıtları gibi temel alanlarda ilişkisel tutarlılığın korunmasıdır. Bu amaçla ASP.NET Core MVC uygulamasında Entity Framework Core ve SQL Server kullanılmakta, varlık sınıfları ile ilişkisel tablolar arasında yönetilebilir bir eşleme kurulmaktadır [10, 11]. İkinci gereksinim ise kullanıcının sabit formların dışında, kendi ihtiyaçlarına göre tablo alanları tanımlayabilmesidir. Dinamik tablo ve hazır tablo yükleme modülleri bu ihtiyaca karşılık gelmekte; CSV/XLSX verilerinin uygulama içine alınması ve satır verilerinin JSON temelli esnek yapılarla tutulması sağlanmaktadır [12].

Bu yaklaşım, tamamen serbest veri girişinin oluşturabileceği dağınıklık ile tamamen katı veritabanı şemasının oluşturabileceği sınırlılık arasında dengeli bir çözüm sunmaktadır. Zaman Sayar, çekirdek iş akışlarında ilişkisel veri bütünlüğünü korurken, dinamik tablo modülüyle kullanıcı tanımlı veri gereksinimlerine de alan açmaktadır.

3.5 OCR destekli bilgi çıkarımı

OCR tabanlı veri çıkarımı, görsel veya belge üzerinde yer alan metnin makine tarafından okunabilir hale getirilmesini sağlayan önemli bir yardımcı teknolojidir. Tesseract OCR motoru, belge işleme alanında uzun süredir kullanılan açık kaynaklı bir çözüm olarak literatürde yer almakta ve farklı belge türlerinden metin çıkarma amacıyla kullanılmaktadır [6, 7]. EasyOCR ise Python ekosistemi içinde çok dilli metin tanıma için pratik bir kullanım sunmakta, özellikle hızlı prototipleme ve uygulama bütünleştirme süreçlerinde tercih edilebilmektedir [8].

Ancak görev yönetimi bağlamında OCR çıktısının yalnızca ham metin olarak elde edilmesi yeterli değildir. Ham metnin görev içeriği, atanan kişi, atayan kişi, son tarih ve açıklama gibi anlamlı alanlara dönüştürülmesi gerekir. Bu nedenle OCR destekli sistemlerde metin tanıma aşaması ile alan çıkarımı aşaması birbirinden ayrılmalıdır. Birinci aşama görselin okunabilir metne dönüştürülmesini, ikinci aşama ise bu metnin uygulama veri modeline uygun hale getirilmesini kapsar.

Zaman Sayar’da OCR bileşeni bu ayrımı açık biçimde uygulamaktadır. Python tabanlı yardımcı servis görsel ön işleme, Tesseract ve EasyOCR ile metin çıkarma, alan eşleştirme ve güven değeri üretme adımlarını yürütmektedir. Elde edilen çıktı doğrudan kaybolan bir metin olarak bırakılmamakta; OCR log kayıtlarıyla izlenebilir hale getirilmekte ve uygun alanlar üzerinden görev taslağına dönüştürülmektedir. Bu yönüyle proje, OCR teknolojisini yalnızca teknik bir eklenti olarak değil, manuel veri girişini azaltan denetlenebilir bir yardımcı katman olarak kullanmaktadır.

3.6 Doğal dil arayüzleri ve güvenli veri erişimi

Doğal dil arayüzleri, kullanıcıların karmaşık sorgu dilleri veya ayrıntılı filtre ekranları kullanmadan veriye erişmesini amaçlayan araştırma alanlarından biridir. Androutsopoulos, Ritchie ve Thanisch, doğal dil veritabanı arayüzlerinin kullanıcı ile veritabanı arasındaki teknik mesafeyi azaltmayı hedeflediğini; ancak anlam belirsizliği, veritabanı şemasıyla eşleştirme ve kullanıcı niyetinin doğru yorumlanması gibi zorluklar içerdiğini belirtmektedir [20]. Li ve Jagadish tarafından önerilen etkileşimli doğal dil arayüzü yaklaşımı da, kullanıcı sorgularının ilişkisel veritabanı yapısıyla anlamlı biçimde ilişkilendirilmesi gerektiğini göstermektedir [21].

Bu çalışmalar, doğal dil arayüzlerinin güçlü tarafının erişim kolaylığı olduğunu; riskli tarafının ise yanlış yorumlanan sorguların veri bütünlüğünü bozabilecek işlemlere dönüşebilmesi olduğunu göstermektedir. Bu nedenle doğal dil ile veri erişimi sağlayan sistemlerde, kullanıcının niyeti kadar sistemin izin verdiği işlem sınırları da açık biçimde tanımlanmalıdır.

Zaman Sayar’daki ZamanChat bileşeni bu riski azaltmak için salt-okunur olarak tasarlanmıştır. Bileşen, veritabanı şemasını okuyarak kullanıcının hangi modül hakkında bilgi istediğini belirlemeye çalışmakta; özetleme, yorumlama veya kayıt ayrıntısı yazma gibi okuma odaklı işlemlerle sınırlı kalmaktadır. Veri ekleme, silme, güncelleme veya tablo değiştirme niyetleri reddedilmektedir. Böylece doğal dil arayüzü kullanıcı deneyimini kolaylaştırmakta, ancak uygulamanın veri bütünlüğü üzerinde kontrolsüz bir yazma yetkisi oluşturmamaktadır.

3.7 Web tabanlı mimari, katmanlı tasarım ve sürdürülebilirlik

Web tabanlı uygulamalarda sürdürülebilirlik için arayüz, iş mantığı, veri erişimi ve yardımcı servislerin birbirinden ayrılması önemlidir. Fowler, kurumsal uygulama mimarilerinde katmanlı yapıların karmaşıklığı yönetmek ve sistemin geliştirilebilirliğini korumak için kullanıldığını vurgulamaktadır [22]. ASP.NET Core MVC yaklaşımı da Model, View ve Controller ayrımıyla bu düşüncüyü desteklemekte; kullanıcı arayüzü, yönlendirme ve veri modeli sorumluluklarının tek bir dosyada birikmesini engellemektedir [9].

Zaman Sayar bu açıdan yalnızca ekranlardan oluşan bir web projesi değildir. Razor görü-

nümleri kullanıcı arayüzünü, controller sınıfları istek akışını, Entity Framework Core veri erişimini, SQL Server kalıcı veri saklamayı, Python OCR servisi görselden görev çıkarmını, Node.js ve Socket.IO tabanlı ZamanChat servisi ise doğal dil ile veri okuma deneyimini sağlamaktadır [15, 16]. Bu yapı, sistemin farklı sorumlulukları ayrı katmanlarda taşımasına olanak vermektedir.

Yazılım kalitesi açısından bakıldığında, ISO/IEC 25010 ürün kalite modeli işlevsel uygunluk, sürdürülebilirlik, kullanılabilirlik, güvenilirlik ve güvenlik gibi özellikleri değerlendirme çerçevesi olarak ele almaktadır [23]. Zaman Sayar'ın mevcut sürümü bitirme projesi kapsamında çalışan bir temel sunmakta; özellikle modülerlik, izlenebilirlik ve geliştirilebilirlik yönleriyle bu kalite başlıklarına karşılık gelen mühendislik kararları içermektedir. Bununla birlikte üretim seviyesinde yetkilendirme, test otomasyonu, hata yönetimi ve kullanıcı deneyimi iyileştirmeleri gelecekte güçlendirilmesi gereken alanlar olarak kalmaktadır.

3.8 Görev metni sınıflandırma ve özetlemede makine öğrenmesi

Kısa iş ve görev metnlerinin sınıflandırılmasında TF-IDF temelli vektörleştirme ile doğrusal modellerin güçlü bir taban çizgisi sunduğu bilinmektedir [24, 27]. Anlamsal benzerliği yakalamak ve yüksek boyutlu terim uzayını daha yönetilebilir hale getirmek için Örtük Anlamsal Analiz yaygın biçimde kullanılmaktadır [25]. Bu yaklaşım, özellikle kısa metinlerde doğrudan anahtar sözcük eşleşmesine dayalı yöntemlerin kaçırabileceği kısmi benzerlikleri yakalamaya yardımcı olur.

Tablo ve yapılandırılmış öznitelik verilerinde ağaç tabanlı artırma yöntemleri yüksek başarımla sağlayabilmektedir. XGBoost ve LightGBM bu alanda sık kullanılan örneklerdir [28, 29]. Çıkarımsal metin özetlemede ise TextRank, metin içindeki cümleleri çizge üzerinde önem derecesine göre sıralayan denetimsiz ve açıklanabilir bir yöntem olarak öne çıkmaktadır [26]. Kümeleme değerlendirmesinde Silhouette, Calinski-Harabasz, Davies-Bouldin, ARI ve NMI gibi ölçütler, kümelerin iç tutarlılığı ve dışsal etiketlerle hizalanması hakkında bilgi verir [30–33].

Zaman Sayar bu literatürdeki yöntemleri tek başına bir makine öğrenmesi çalışması olarak değil, çalışan bir CRM/zaman yönetimi ürünü içine bağlanan karar destek katmanı olarak kullanmaktadır. Bu yönüyle sistem; Türkçe görev metninden tür, kategori, öncelik, süre, RACI rolü ve özet üretme hedeflerini aynı uygulama bağlamında ölçülebilir biçimde birleştirmektedir.

3.9 Literatürden türetilen tasarım çıkarımları

Kaynak araştırması sonucunda Zaman Sayar için öne çıkan temel tasarım çıkarımları Tablo 3 üzerinde özetlenmiştir. Bu tablo, literatürdeki problem alanlarını doğrudan proje içindeki

modüllerle ilişkilendirmektedir.

Tablo 3: Literatürden türetilen tasarım çıkarımları

Literatür alanı	Öne çıkan gereksinim	Zaman Sayar'daki karşılığı
Görev yönetimi	İşlerin yalnızca listelenmemesi, öncelik ve durum bilgisiyle izlenmesi	Görev/Görevler ve Görev Atama modülleri
Takvim kullanımı	İşlerin zaman ekseninde görünür hale getirilmesi	Gün, hafta ve ay görünümlü takvim yapısı
Kişisel bilgi yönetimi	Not, ses kaydı ve destekleyici bilginin daha sonra bulunabilir kalması	Not, sesli not, kategori ve etiket alanları
Sorumluluk atama	Görev çevresindeki rollerin yalnızca atanmış kişiyle sınırlı kalmaması	RACI matrisi, rol alanları ve yorumlar
Veri modelleme	Kayıtların ilişkili, tutarlı ve izlenebilir saklanması	Entity Framework Core, SQL Server ve ilişkili tablolar
Esnek veri girişi	Kullanıcının sabit formların dışında veri saklayabilmesi	Dinamik Tablo ve Hazır Tablo Yükle modülleri
OCR teknolojileri	Görselden alınan metnin uygulama alanlarına dönüştürülmesi	OCR servisinin görev alanı çıkarımı ve log kaydı
Doğal dil arayüzleri	Veriye hızlı erişim sağlanırken veri bütünlüğünün korunması	Salt-okunur ZamanChat sorgu katmanı
Makine öğrenmesi	Kısa görev metninden sınıflandırma, tahmin ve özet üretme	zaman_ai ile akıllı öneri ve ölçülebilir karar destek katmanı

3.10 Zaman Sayar'ın konumlandırılması

Literatür ve mevcut araç yaklaşımları birlikte değerlendirildiğinde, Zaman Sayar'ın ana farkı tek bir modülde uzmanlaşmasından değil, birden fazla işlevi aynı veri bağlamı içinde birleştirmesinden kaynaklanmaktadır. Mevcut dijital araçların önemli bir bölümü takvim, görev listesi, not alma, tablo düzenleme veya sohbet tabanlı sorgulama gibi belirli bir işleve odaklanmaktadır. Bu tür araçlar kendi alanlarında güçlü olmakla birlikte, kullanıcı farklı iş akışlarını aynı proje bağlamında yürütmek istediğinde veri parçalanması ve tekrar giriş ihtiyacı ortaya çıkabilmektedir.

Zaman Sayar bu noktada bütünsel bir çalışma alanı yaklaşımı sunmaktadır. Görevler zamanla, notlar görev bağlamıyla, RACI kayıtları sorumluluk yapısıyla, OCR çıktıları görev taslağıyla, dinamik tablolar ise esnek veri ihtiyacıyla ilişkilendirilmektedir. Tablo 4 bu konumlandırmayı mevcut araç yaklaşımlarıyla karşılaştırmalı olarak özetlemektedir.

Tablo 4: Mevcut araç yaklaşımları ile Zaman Sayar'ın karşılaştırılması

Yaklaşım	Güçlü yön	Tek başına sınırlılığı	Zaman Sayar'ın eklediği değer
Takvim uygulamaları	Zaman görünürlüğü ve randevu takibi	Görev ayrıntısı, sorumluluk ve not bağlamı sınırlı kalabilir	Görev, not, RACI ve istatistik ile aynı bağlamda zaman planlama
Görev listesi araçları	Yapılacak işlerin durum ve öncelikle izlenmesi	Zaman, OCR, tablo ve sorumluluk yapısı ayrı araçlara dağılabilir	Alt görev, görev atama, OCR ve takvim bağlantısı
Not uygulamaları	Serbest metin ve hızlı bilgi kaydı	Notlar çoğu zaman iş akışından kopuk kalabilir	Notların görev ve çalışma akışıyla aynı sistemde tutulması
Elektronik tablo araçları	Esnek veri düzenleme ve listeleme	Kullanıcı deneyimi görev ve zaman yönetimine doğrudan bağlı değildir	Dinamik tablo ve hazır tablo yükleme akışının uygulama içinde yer alması
RACI şablonları	Sorumlulukların rol bazlı ayrıştırılması	Çoğu zaman statik doküman veya ayrı dosya olarak kalır	RACI bilgisinin canlı görev kayıtları ve yorumlarla birlikte yönetilmesi
OCR araçları	Görselden ham metin çıkarımı	Çıkarılan metnin uygulama alanlarına dönüşmesi ayrıca gerekir	Ham metnin görev alanlarına dönüştürülmesi ve denetim kaydı tutulması
Sohbet arayüzleri	Bilgiye doğal dil ile erişim	KontROLSÜZ yazma yetkisi veri bütünlüğü riski oluşturabilir	Salt-okunur, şema farkında ve sınırlı eylem modeli
Makine öğrenmesi araçları	Sınıflandırma ve tahmin desteği	Ürün iş akışından kopuk kalabilir veya yalnızca çevrimdışı deney olarak kalabilir	Akıllı öneri çıktılarının canlı arayüz, geri bildirim ve ölçüm tablosuyla bağlanması

4 MATERYAL VE YÖNTEM

4.1 Geliştirme yaklaşımı

Zaman Sayar, gereksinimlerin modüllere ayrıldığı, her modülün kendi veri modeli ve arayüz akışıyla geliştirildiği, daha sonra ortak ana sayfa ve ortak veritabanı üzerinden bütünleştirildiği artımlı bir geliştirme yaklaşımıyla hazırlanmıştır. Çalışmada öncelikle çekirdek görev ve takvim akışları oluşturulmuş, ardından not, RACI, dinamik tablo, hazır tablo yükleme, istatistik, OCR ve ZamanChat bileşenleri sisteme eklenmiştir. Bu yaklaşım, projenin yalnızca tek bir ekran olarak değil, birbirini tamamlayan yazılım bileşenleri bütünü olarak ele alınmasını sağlamıştır.

Yöntemsel olarak çalışma üç eksende yürütülmüştür. Birinci eksen, kullanıcı gereksinimlerinin modüllere dönüştürülmesidir. İkinci eksen, bu modülleri taşıyacak katmanlı yazılım mimarisinin kurulmasıdır. Üçüncü eksen ise uygulama çıktılarının arayüz, veri modeli, algoritma ve sınırlılık açısından değerlendirilmesidir.

4.2 Sistem gereksinimleri

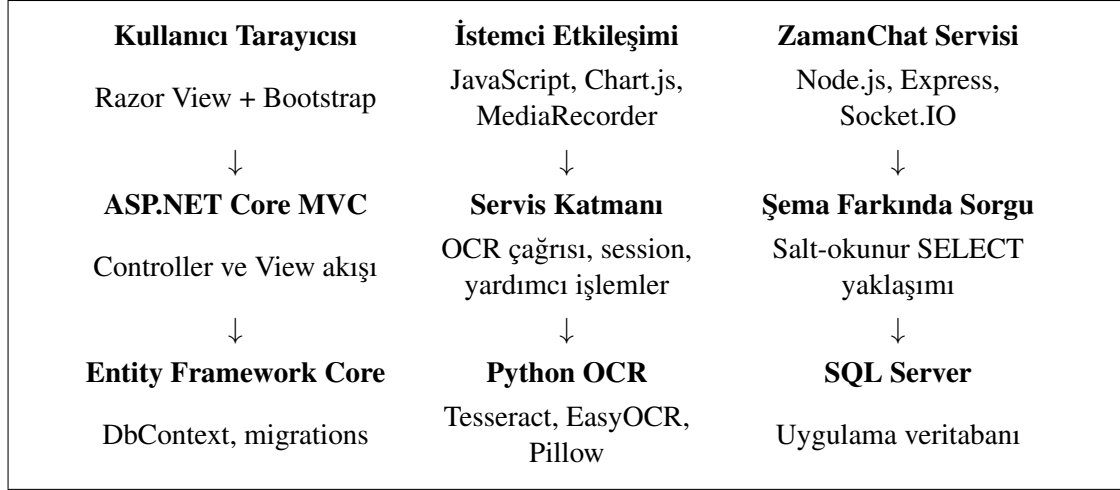
Sistemin işlevsel gereksinimleri, kullanıcının bir iş kaydını oluşturma, zamanla ilişkilendirme, notlarla destekleme, sorumluluk paylaşımını tanımlama, yapılandırılmış veriyle çalışma ve kayıtları daha sonra yorumlayabilme ihtiyacından türetilmiştir. İşlevsel olmayan gereksinimler ise sürdürülebilirlik, modülerlik, izlenebilirlik, veri bütünlüğü ve geliştirilebilirlik üzerinde yoğunlaşmaktadır.

Tablo 5: Zaman Sayar sistem gereksinimleri

Gereksinim sınıfı	Gereksinim	Karşıllayan bileşen
Planlama	Görevlerin tarih ve zaman bağlamında izlenmesi	Takvim, günlük plan panosu
Görev takibi	Durum, öncelik, atanan kişi ve alt görev bilgisi	Görev/Görevler, Görev Atama
Bilgi kaydı	Yazılı ve sesli notların saklanması	Not modülü, MediaRecorder desteği
Sorumluluk yönetimi	R, A, C ve I rollerinin ayrıştırılması	RACI matrisi ve rol filtreleri
Esnek veri	Kullanıcı tanımlı tablo ve dosya aktarımı	Dinamik Tablo, Hazır Tablo Yükle
Otomasyon	Görselden görev alanı üretimi	Python OCR servisi
Bilgi erişimi	Doğal dil ile kayıt okuma	ZamanChat salt-okunur servisi
Görselleştirme	Tamamlama ve dağılım bilgilerinin izlenmesi	İstatistik ekranları, Chart.js

4.3 Yazılım mimarisi

Uygulama katmanlı bir web mimarisi üzerinde çalışmaktadır. Ana uygulama ASP.NET Core MVC ile geliştirilmiş, Razor görünümleri üzerinden kullanıcı arayüzü üretilmiş, veritabanı işlemleri Entity Framework Core aracılığıyla SQL Server üzerinde gerçekleştirilmiştir [9–11]. OCR ve ZamanChat bileşenleri çekirdek web uygulamasına yardımcı servisler olarak bağlanmıştır. Bu yapı Şekil 9 üzerinde gösterilmektedir.



Şekil 9: Zaman Sayar yazılım mimarisi

Tablo 6: Mimaride kullanılan başlıca teknolojiler

Katman	Teknoloji	Kullanım amacı
Web uygulaması	ASP.NET Core MVC, Razor	Controller tabanlı iş akışı ve sunucu tarafı sayfa üretimi
Veri erişimi	Entity Framework Core 9, SQL Server	Entity sınıflarının ilişkisel veritabanında saklanması
İstemci tarafı	Bootstrap, JavaScript, Chart.js	Arayüz etkileşimi ve grafik görselleştirme [13, 14]
Dosya işlemleri	EPPlus, CSV işleme	Hazır tabloların yüklenmesi ve dışa aktarılması [12]
OCR	Python, Pillow, Tesseract, EasyOCR	Görsel ön işleme ve metin çıkarımı
Sohbet servisi	Node.js, Express, Socket.IO, MSSQL sürücüsü	Doğal dil ile salt-okunur veri sorgulama
Makine öğrenmesi	Python, scikit-learn, pandas, NumPy, joblib	Görev sınıflandırma, süre tahmini, RACI önerisi, özetleme ve deney kaydı
GitHub entegrasyonu	GitHub REST API, HttpClientFactory	Depo, konu, pull request, etkinlik ve takvim senkronu verilerinin alınması

4.4 Proje yapısı

Kaynak kodu dört ana .NET projesi ve iki yardımcı servis alanı üzerinden düzenlenmiştir. CRM_9 web uygulamasını, controller sınıflarını, Razor görünümelerini, servisleri ve statik dosyaları içerir. CRM.Models entity sınıfları ile view model yapılarını tutar. CRM.DataAccess veritabanı bağlamını ve migration dosyalarını içerir. CRM.Utility ortak sabit ve yardımcı yapıların tutulduğu projedir. Yardımcı bileşenler olarak Python/ocr_to_task.py OCR iş akışını, NodeBot/server.js ise ZamanChat servisini yürütmektedir.

Tablo 7: Controller ve sorumluluk haritası

Controller	Temel sorumluluk
HomeController	Ana sayfa ve günlük plan panosu için genel görünüm.
TaskController	Kişisel görevler, alt görevler, durum ve soft-delete akışı.
TaskAssignmentController	Ayrıntılı görev atama ve OCR ile görselden görev oluşturma.
CalendarController	Gün, hafta ve ay bağlamında takvim görevleri.
NoteController	Yazılı not, kategori, etiket ve sesli not işlemleri.
DynamicBoardController	RACI matrisi, yorumlar ve rol bazlı özetler.
DynamicTableController	Kullanıcı tanımlı dinamik tablo ve JSON satır verisi.
DynamicTablesController	Excel/CSV yükleme, başlık güncelleme, satır ekleme ve dışa aktarma.
StatisticsController	Tamamlama, dağılım ve yoğun gün istatistikleri.
SuggestionController	zaman_ai modelinden akıllı öneri alma ve kullanıcı geri bildirimlerini kaydetme.
GitHubController	GitHub bağlantı ayarları, depo/konu/PR görünümü, gösterge paneli ve takvim senkronu.

4.5 Veri modeli

Veri modeli, Identity tabanlı kullanıcı altyapısı ve uygulamaya özgü modül tablolarından oluşmaktadır. Görev atama kayıtları OCR logları ile ilişkilendirilebilmekte, görev öğeleri kendi içinde alt görev ilişkisi kurabilmekte, RACI panosu yorumlarla genişleyebilmekte, dinamik tablolar ve hazır tablo yükleme akışları ise JSON tabanlı satır verisi saklamaktadır. Bu yapı, hem ilişkisel tutarlılığı hem de esnek veri girişini desteklemektedir.

Tablo / Varlık	İlişki ve görev
TaskAssignments	Ayrıntılı görev atama kayıtlarını tutar; OcrTaskLogs ile 1-N ilişki kurabilir.
OcrTaskLogs	OCR motoru, ham metin, güven değeri ve oluşturulan görev izini saklar.
TaskItems	Kişisel görevleri ve ana-alt görev ilişkisini içerir.
CalendarTasks	Zaman planlama, tekrar, hatırlatma ve renk etiketi bilgilerini saklar.
Notes	Metin, kategori, etiket ve sesli not yolunu tutar.
DynamicBoardItems	RACI görevlerini, rol dağılımını ve ilerleme yüzdesini saklar.
DynamicBoardComments	RACI kayıtlarına bağlı yorumları tutar.
DynamicTableDefinitions / DynamicTableData	Kullanıcı tanımlı tablo alanları ve JSON satır verisi için kullanılır.
UploadedTables / UploadedRows	Excel/CSV dosya kökü ve aktarılan JSON satır verilerini saklar.

Şekil 10: Zaman Sayar veri modeli özeti

4.6 Modül tasarımı

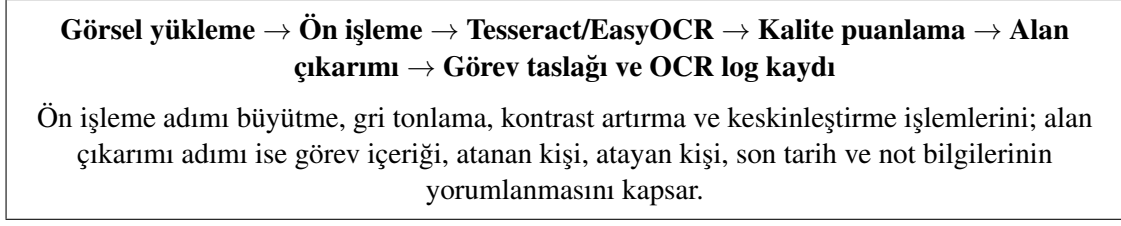
Uygulamada her modül belirli bir kullanıcı ihtiyacına karşılık gelmektedir. Takvim modülü zaman eksenini, görev modülü iş parçalarını, görev atama modülü resmi takip kayıtlarını, not modülü bağlam bilgisini, RACI modülü sorumluluk paylaşımını, tablo modülleri esnek veri girişini, istatistik modülü yorumlanabilir çıktıyı, OCR ve ZamanChat ise yardımcı otomasyon temsil etmektedir. Modüllerin tasarımında ortak ilke, kullanıcının tek panelden çoklu iş akışını yönetebilmesidir.

Tablo 8: Modül bazlı tasarım özeti

Modül	Ana veri yapısı	Mühendislik değeri
Takvim	CalendarTask	Görevleri tarih, tekrar, hatırlatma ve renk etikleyle izler.
Görev/Görevler	TaskItem	Alt görev, tamamlanma, hatırlatma ve soft-delete desteği sağlar.
Görev Atama	TaskAssignment	Atayan, atanan, öncelik, takip ve tamamlanma bilgilerini tutar.
Not	Note	Metin, kategori, etiket ve sesli not verisini aynı kayıt altında saklar.
RACI	DynamicBoardItem	R, A, C, I rollerini ve ilerleme yüzdesini görünür kılar.
Dinamik Tablo	DynamicTableDefinition, DynamicTableData	Kullanıcı tanımlı alanlarla esnek tablo yapısı sunar.
Hazır Tablo Yükle	UploadedTable, UploadedRow	CSV/XLSX dosyalarını JSON satır verisine dönüştürür.
İstatistik	StatisticsViewModel	Görev dağılımı ve yoğunluk bilgisini grafikleştirir.
OCR	OcrTaskLog, OcrTaskExtractionResult	Görselden görev alanı üretir ve ham çıktıyı izlenebilir kılar.
ZamanChat	Canlı SQL şema okuma	Kayıtları doğal dille ve salt-okunur şekilde özetler.
GitHub	GitHubSetting, CalendarTask	Depo, konu, PR ve takvim senkronu akışını uygulama bağlamına bağlar.
Akıllı Öneri	SuggestionFeedback, TaskSuggestionResult	Görev metninden ML destekli tür, kategori, öncelik, süre, RACI ve özet önerisi üretir.

4.7 OCR iş akışı

OCR bileşeni iki aşamalı tasarlanmıştır. Birinci aşamada görsel hazırlanmakta ve Tesseract ile EasyOCR motorlarından metin çıkarılmaktadır. İkinci aşamada ham metin, Türkçe ve İngilizce etiket eş anlamlıları yardımıyla görev alanlarına dönüştürülmektedir. Sistem, yalnızca en uzun metni değil, görev alanı çıkarımı açısından en kullanışlı sonucu seçmeye çalışmaktadır.



Şekil 11: OCR destekli görev çıkarımı iş akışı

Motor seçimi kavramsal olarak Denklem 1 ile ifade edilebilir. Burada F_m çıkarılabilen görev alanı sayısını, E_m etiket eşleşme başarısını, Q_m ham metin okunabilirliğini, C_m ise motorun güven değerini temsil etmektedir.

$$S_m = 0.35F_m + 0.25E_m + 0.20Q_m + 0.20C_m \quad (1)$$

Uygulamadaki Python betiği bu kavramı doğrudan tek formüle indirgmeden, alan varlığı, yapılandırılmış etiket sayısı, Türkçe karakter yoğunluğu, metin bozukluğu ve motor güveni gibi ölçütleri birlikte değerlendirir. Bu yaklaşım, OCR çıktısının yalnızca teknik olarak okunmuş olmasını değil, görev üretimi için kullanılabilir olmasını önemser.

4.8 ZamanChat iş akışı ve güvenlik sınırı

ZamanChat, uygulamadaki veriye doğal dil ile erişimi kolaylaştırmak için geliştirilmiş yardımcı bir bileşendir. Servis, veritabanı şemasını INFORMATION_SCHEMA üzerinden okuyarak tablo ve kolon bilgilerini çıkarır. Kullanıcı metninde sekme adı, işlem tipi ve kayıt kimliği aranır. İşlem tipi yalnızca özetleme, yorumlama veya bütün bilgileri yazma seçenekleriyle sınırlıdır. Veri ekleme, silme, güncelleme veya tablo değiştirme niyeti algılandığında servis işlem yapmaz ve kullanıcıya salt-okunur çalıştığını bildirir.

Tablo 9: ZamanChat karar akışı

Adım	İşlem
Girdi temizleme	Kullanıcı metni normalize edilir, boş girdi reddedilir.
Yazma niyeti denetimi	Ekleme, silme, güncelleme, düşürme veya değiştirme anlamı taşıyan ifadeler yakalanır.
Eylem belirleme	“Özetle”, “yorumla” veya “bütün bilgilerini yaz” eylemlerinden biri aranır.
Profil belirleme	Takvim, Görev Atama, Not, RACI, Dinamik Tablo gibi sekme adları eşleştirilir.
Kayıt seçimi	Takvim için tarih, diğer profiller için ID bilgisi beklenir.
Sorgu çalıştırma	Parametrelili ve sınırlandırılmış SELECT sorgusu üretilir.
Yanıt üretimi	Sonuçlar kullanıcıya özet, yorum veya ayrıntılı bilgi formatında döndürülür.

4.9 GitHub Entegrasyonu

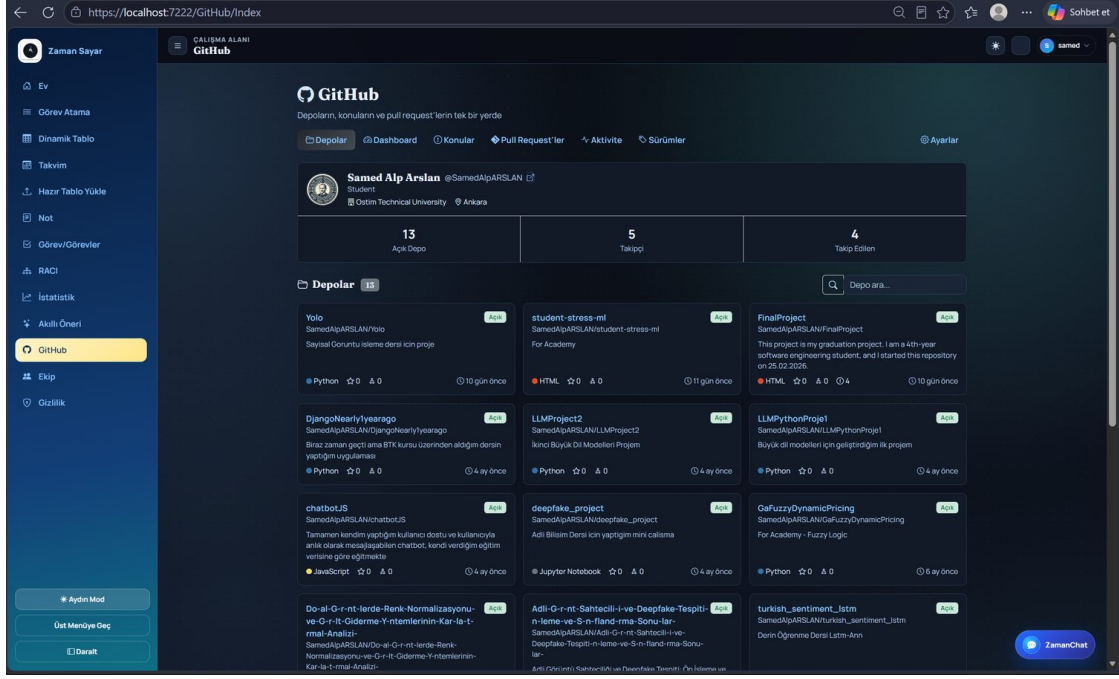
Yazılım geliştirme ekiplerinin görev yönetimini, kodun barındırıldığı sürüm kontrol platformuyla aynı arayüzde birleştirmek amacıyla sisteme bir *GitHub entegrasyonu* eklenmiştir (Şekil 12). Entegrasyon, GitHub REST API'si (`api.github.com`, API sürümü 2022-11-28) üzerinden, kullanıcı/takım başına saklanan bir kişisel erişim belirteci (Personal Access Token, PAT) ile yetkilendirilir. Tüm çağrılar, mevcut çok kiracılı mimariye uygun olarak `OwnerId` (takım) anahtarıyla ilişkilendirilir ve `IHttpClientFactory` ile yönetilen bir `HttpClient` üzerinden, Bearer yetkilendirme ve GitHub'ın zorunlu kıldığı `User-Agent` başlığıyla yapılır.

Entegrasyon, artımlı olarak üç aşamada tasarlanmıştır:

1. **Profil ve depolar:** Bağlanan hesabın profili ile depo listesi, en son güncellenenler üstte olacak biçimde çekilir ve kart düzeninde sunulur.
2. **Konular (Issues) ve Pull Request'ler:** GitHub Arama API'si kullanılarak kullanıcının atandığı, yazdığı veya anıldığı konular ve pull request'ler süzülür. Ayrıca, kilometre taşı (milestone) bitiş tarihi (`due_on`) bulunan açık konular, uygulamanın kendi *takvimine* (`CalendarTask`) yinelenmesiz biçimde senkronize edilebilir; böylece kod tarafındaki son teslim tarihleri görev takviminde de görünür hale gelir.
3. **Gösterge paneli, etkinlik ve sürümler:** Depo sayısı, yıldız, çatallama (fork) ve açık konu toplamaları gibi temel başarımlar göstergeleri (KPI); programlama dili dağılımı; en çok yıldızlı ve en son güncellenen depolar; son 14 günün günlük etkinlik grafiği ile depolardaki sürüm (release) kayıtları derlenip özetlenir.

Hata dayanıklılığı için tüm API çağrıları savunmacı biçimde sarmalanmıştır: geçersiz belirteç veya ağ hatası durumunda kullanıcıya bilgilendirici bir ileti gösterilir ve boş sonuç döndürülerek arayüzün çökmesi önlenir. Veritabanı bağlamının (`DbContext`) iş parçacığı güvenli olmaması nedeniyle, gösterge panelini besleyen çoklu çağrılar bilinçli olarak sıralı yürütülür; bu tercih doğruluk ile yanıt süresi arasında ölçülü bir ödünleşimdir.

Bilinen sınırlama (dürüst raporlama). Kişisel erişim belirteci, bu sürümde veritabanında şifrelenmeden saklanmaktadır. Üretim ortamı için belirtecin ASP.NET Core *Data Protection* API'si veya bir gizli yönetimi servisi (örn. Azure Key Vault) ile şifrelenerek saklanması gerekir; bu iyileştirme planlanan gelecek iş kapsamındadır.



Şekil 12: Uygulama içi GitHub entegrasyonu: bağlanan hesabın profili, depo sayıları ve dil etiketleriyle birlikte depo kartları. Üst sekmeler depo, gösterge paneli, konular, pull request, etkinlik ve sürüm görünümüne erişim sağlar.

4.10 Makine öğrenmesi alt sistemi (zaman_ai)

Projeye, mevcut çekirdek mimari değiştirilmeden bağımsız bir Python paketi olan zaman_ai eklenmiştir. Paket; ön işleme, veri üretimi, öznitelik çıkarımı, sınıflandırma (A1), süre/gecikme tahmini (A2), özetleme (A3), RACI önerisi (A4), deney kaydı ve kümeleme (EK-3) modüllerinden oluşur. C# tarafı, OCR bileşeniyle aynı süreç sözleşmesini kullanır: predict.py son satırında tek bir JSON nesnesi üretir, ASP.NET Core servis katmanı bu çıktıyı uygulama modeline dönüştürür. İlk çağrıda modeller yoksa otomatik eğitilir ve diske kaydedilir; sonraki çağrılarda kayıtlı modeller yüklenerek çıkarım yapılır. Tekrarlanabilirlik için tüm rastgelelik sabit tohumla (42) kontrol edilmiştir.

Tablo 10: zaman_ai modülleri ve araştırma hedefleri

Modül	Hedef	Görev
text_processing.py	Ortak	Türkçe metin temizleme, normalleştirme ve hafif kök/biçim sadeleştirme.
dataset.py	A5	Sentetik ve anonim Türkçe görev veri kümesi üretimi.
features.py	A1–A4	TF-IDF, LSA ve zaman/iş yükü özniteliklerinin çıkarılması.
classifier.py	A1	Görev türü, kategori ve öncelik sınıflandırması.
deadline.py	A2	Süre tahmini, hatırlatma zamanı ve gecikme azaltımı benzetimi.
summarizer.py	A3	TextRank tabanlı çıkarımsal özetleme ve okunabilirlik değerlendirmesi.
raci.py	A4	RACI rol önerisi ve kabul oranı ölçümü.
clustering.py	EK-3	Görev metni kümeleme ve dışsal hizalanma metrikleri.
run_experiments.py	A5	Tüm deneyleri çalıştırıp CSV/JSON/Markdown ölçüm raporu üretme.

4.11 Veri kümesi ve etik

Modeller, gerçek kişisel veri kullanılmaması ilkesiyle sentetik ve anonim bir Türkçe görev veri kümesi üzerinde geliştirilmiştir. Veri kümesi 1200 kayıt içerir; kişi adı yerine gelistirici, test_uzmani, tasarimci, analist, destek_uzmani, yönetici ve ekip_lideri gibi rol etiketleri kullanılır. Her kayıt; görev metni, tür, kategori, öncelik, oluşturma/bitiş/tamamlanma tarihleri, tahmini ve gerçek süre, atanan rol, iş yükü, RACI rolü ve gecikme gibi alanlar taşır.

Veri, kolayca ayrılan yapay bir örnek olmaması için orta zorlukta öğrenilebilir bir rejime kalibre edilmiştir. Böylece modellerin tamamen kusursuz sonuç üretmesi engellenmiş, başarı ve sınırlılıklar birlikte görülebilir hale getirilmiştir. A5 kapsamında ölçüm raporları, açık ER şeması ve anonim örnek veri dosyaları korunmuştur. Bu yaklaşım, KVKK açısından gerçek kişi verisini modelleme sürecinin dışında tutarken deneylerin tekrarlanabilirliğini de desteklemektedir.

4.12 Öznitelik çıkarımı ve modeller

Metin öznitelikleri TF-IDF ile çıkarılmıştır. Bu aşamada 1–2 gram aralığı, min_df=2, max_df=0.95, sublinear_tf ve en çok 5000 öznitelik sınırı kullanılmıştır. Anlam-sal vekil temsil için TruncatedSVD tabanlı LSA uygulanmıştır. Zaman temelli öznitelikler; tahmini süre, iş yükü, öncelik sırası, haftanın günü, hafta sonu göstergesi ve tür kodlaması gibi alanlardan oluşur.

Sınıflandırmada üç yaklaşım karşılaştırılmıştır: kural temeli, lojistik regresyon ve histogram

tabanlı gradyan artırma. Lojistik regresyonda dengeli sınıf ağırlığı kullanılmıştır. Ağaç tabanlı modellerde XGBoost veya LightGBM kuruluysa bu modeller tercih edilebilir; aksi durumda scikit-learn içindeki HistGradientBoosting kullanılmaktadır. Süre tahmini regresyon problemi, RACI önerisi ise çok sınıflı sınıflandırma problemi olarak değerlendirilmiştir.

4.13 Değerlendirme protokolü

Tüm modeller 5-kat tabakalı çapraz doğrulama ile değerlendirilmiştir. Sınıflandırmada doğruluk, makro-F1, ağırlıklı F1 ve ROC-AUC; regresyonda MAE, MAPE ve R^2 ; kümelemede Silhouette, Calinski-Harabasz, Davies-Bouldin, ARI ve NMI ölçütleri raporlanmıştır. Modeller arası farkın yorumlanabilmesi için eşli t-testi, Wilcoxon işaretli sıra testi, Cohen's d etki büyüklüğü ve %95 güven aralığı bilgileri ölçüm raporunda saklanmıştır. Deney çıktıları CSV, JSON ve Markdown biçimlerinde korunarak sonuçların tekrar denetlenebilir olması hedeflenmiştir.

4.14 Değerlendirme göstergeleri

Uygulamanın değerlendirilmesinde yalnızca ekranların varlığı değil, bu ekranların hangi ölçülebilir işlevleri desteklediği de dikkate alınmıştır. Görev tamamlama oranı Denklem 2 ile ifade edilebilir.

$$T = \frac{N_{tamamlanan}}{N_{toplam}} \times 100 \quad (2)$$

Belirli bir gün için iş yoğunluğu, o güne düşen görev sayısı ve öncelik ağırlıklarıyla birlikte Denklem 3 biçiminde modellenir. Burada $w(p_i)$, görevin önceliğine verilen ağırlığı; c_i , görevin tamamlanma durumunu göstermektedir.

$$Y(d) = \sum_{i=1}^n I(tarih_i = d) \cdot w(p_i) \cdot (1 - c_i) \quad (3)$$

RACI rol kapsamı ise Denklem 4 ile gösterilebilir. r_R , r_A , r_C ve r_I ilgili rol alanının dolu olması durumunda 1, boş olması durumunda 0 değerini alır.

$$C_{RACI} = \frac{r_R + r_A + r_C + r_I}{4} \quad (4)$$

Makine öğrenmesi sonuçlarının yorumlanmasında makro-F1, sınıf dengesizliğinden daha az etkilenmek için sınıf başına F1 değerlerinin ortalaması olarak kullanılmıştır:

$$F1_{macro} = \frac{1}{K} \sum_{k=1}^K \frac{2P_k R_k}{P_k + R_k} \quad (5)$$

Süre tahmininde ortalama mutlak yüzde hata (MAPE) Denklem 6 ile, gecikme azaltımı ise Denklem 7 ile ifade edilmiştir.

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (6)$$

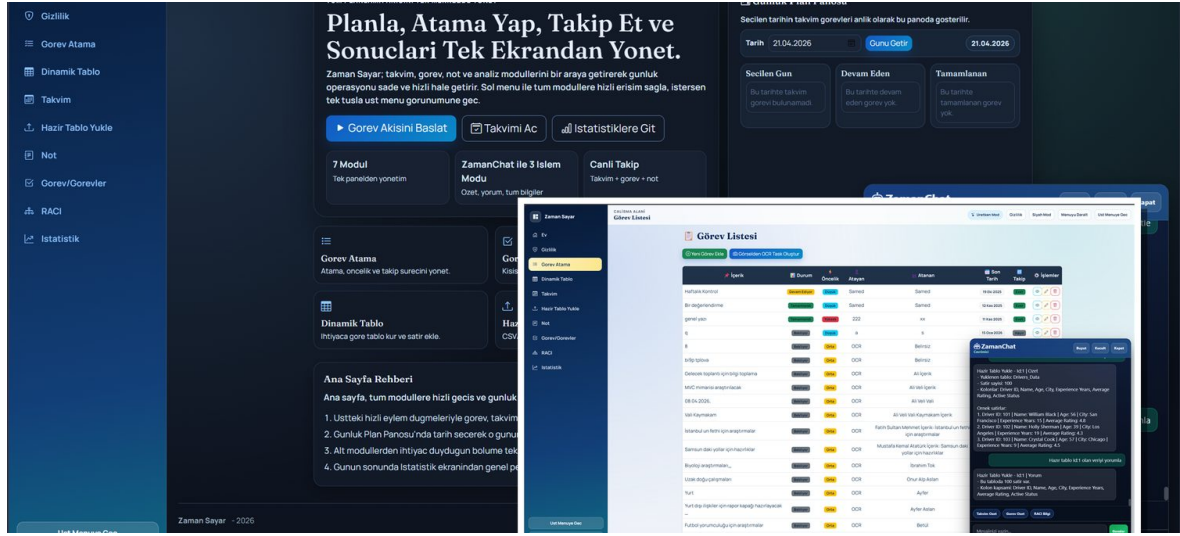
$$\Delta = 100 \times \frac{L_{taban} - L_{model}}{L_{taban}} \quad (7)$$

A4 hedefindeki kabul oranı, çapraz-doğrulama-dışı tahminin onaylı RACI etiketiyle uyuşma yüzdesi olarak tanımlanmıştır. Bu göstergeler, uygulamanın yalnızca kayıt tutan bir sistem değil, kaydedilen bilgiyi yorumlanabilir hale getiren bir karar destek altyapısı olarak da kullanılabileceğini göstermektedir.

5 ARAŞTIRMA SONUÇLARI VE TARTIŞMA

5.1 Uygulama çıktıları

Çalışma sonucunda Zaman Sayar adı verilen, çok modüllü ve web tabanlı bir zaman-görev yönetimi uygulaması elde edilmiştir. Uygulamanın ana ekranı, kullanıcıyı tek bir çalışma panelinde karşılamakta; sol menü üzerinden görev atama, dinamik tablo, takvim, hazır tablo yükleme, not, görev/görevler, RACI ve istatistik modüllerine erişim sağlamaktadır. Ana sayfada ayrıca günlük plan panosu ve ZamanChat bileşeni yer almaktadır. Bu yapı, projenin yalnızca belirli bir ekranı değil, bütünlük bir ürün akışını hedeflediğini göstermektedir.



Şekil 13: Zaman Sayar ana paneli, görev listesi ve ZamanChat görünümü

Uygulama kaynak kodu yerel geliştirme ortamında .NET 9 SDK ile derlenmiş, derleme 0 hata ve 0 uyarı ile tamamlanmıştır. Bu durum sistemin çalıştırılabilir ve teknik olarak tutarlı bir temel sunduğunu göstermektedir. Bununla birlikte üretim kalitesine yaklaşmak için model null güvenliği, form doğrulama, hata izleme ve kullanıcı deneyimi tarafındaki kalite güvence adımlarının ayrıca sürdürülmesi gerekmektedir.

5.2 Fonksiyonel doğrulama

Fonksiyonel değerlendirme, her modülün beklenen temel görevi yerine getirip getirmediği ve uygulama bütünlüğüne nasıl katkı sağladığı üzerinden yapılmıştır. Tablo 11, geliştirilen modüllerin gözlenen çıktıları özetlemektedir.

Tablo 11: Fonksiyonel doğrulama özeti

Modül	Doğrulanana işlev	Değerlendirme
Ana panel	Modüllere hızlı erişim ve günlük plan panosu	Kullanıcıyı tek merkezden yönlendiren bütünleşik giriş noktası oluşmuştur.
Görev Atama	Atayan, atanan, öncelik, tarih ve takip alanları	Resmi görev takibi için ayrıntılı kayıt yapısı sağlanmıştır.
Görev/Görevler	Alt görev, tamamlanma ve hatırlatma	Kişisel iş parçalama ve izleme akışı desteklenmiştir.
Takvim	Aylık, haftalık ve günlük zaman görünümü	Görevlerin zaman bağlamı görünür hale getirilmiştir.
Not	Metin, kategori, etiket ve sesli not	Çalışma bağlamı görevlerden bağımsız fakat aynı sistem içinde saklanabilmektedir.
RACI	R, A, C, I rolleri ve yorumlar	Sorumluluk paylaşımı görev kayıtlarından ayrı bir görünürlük kazanmıştır.
Dinamik tablo	Kullanıcı tanımlı alan ve JSON satır verisi	Esnek veri saklama ihtiyacı karşılanmıştır.
Hazır tablo	Excel/CSV yükleme ve dışa aktarma	Dış veri kaynakları uygulama içine alınabilir hale gelmiştir.
OCR	Görselden görev alanı çıkarımı	Manuel veri girişini azaltan yardımcı otomasyon sağlanmıştır.
ZamanChat	Salt-okunur doğal dil sorgulama	Kullanıcı veriye daha hızlı erişebilmekte, yazma işlemleri engellenmektedir.
GitHub	Depo, konu, PR, etkinlik ve takvim senkronu	Yazılım geliştirme izleri görev-zaman yönetimi bağlamına bağlanmıştır.
Akıllı Öneri	Görev metninden ML tabanlı öneri üretimi	Tür/kategori/öncelik, süre, RACI ve özet çıktıları tek arayüzde sunulmuştur.

5.3 A1: Görev türü, kategori ve öncelik sınıflandırması

Makine öğrenmesi alt sisteminde A1 hedefi kapsamında görev türü, kategori ve öncelik alanları ayrı sınıflandırma görevleri olarak ölçülmüştür. Tablo 12, 1200 sentetik ve anonim kayıt üzerinde 5-kat çapraz doğrulama sonuçlarını göstermektedir.

Tablo 12: A1 – Sınıflandırma başarımı (1200 kayıt, 5-kat çapraz doğrulama)

Hedef	Model	Doğruluk	Makro-F1	Ağırlıklı F1	ROC-AUC
Tür	Kural temeli	0,8342	0,8408	0,8388	0,500
Tür	LogReg	0,9117	0,9116	0,9118	0,9926
Tür	HistGB	0,9042	0,9046	0,9043	0,9869
Kategori	Kural temeli	0,9058	0,9097	0,9068	0,500
Kategori	LogReg	0,9183	0,9163	0,9183	0,9933
Kategori	HistGB	0,9183	0,9162	0,9182	0,9879
Öncelik	Kural temeli	0,8750	0,8766	0,8727	0,500
Öncelik	LogReg	0,9308	0,9273	0,9313	0,9941
Öncelik	HistGB	0,9142	0,9129	0,9142	0,9884

Üç hedefte de en iyi model Lojistik Regresyon olmuştur. Makro-F1 değerleri 0,91–0,93 aralığında gerçekleşmiş ve 0,85 eşliğini açık biçimde aşmıştır. Tür hedefinde Lojistik Regresyon ile kural temeli karşılaştırıldığında ortalama F1 farkı 0,071; eşli t-testi $p = 0,00248$; Cohen's $d = 3,03$ ve %95 güven aralığı $[0,898; 0,925]$ bulunmuştur. Bu sonuç, iyileşmenin istatistiksel olarak anlamlı olduğunu göstermektedir.

5.4 A2: Süre tahmini ve gecikme azaltımı

A2 hedefi, görev süresini tahmin etmek ve bu tahmini kullanarak gecikmeyi azaltmayı amaçlamaktadır. Tablo 13, süre tahmini başarımını; Tablo 14, bütçe-eşleştirilmiş gecikme benzetimini özetlemektedir.

Tablo 13: A2 – Süre tahmini başarımı (5-kat çapraz doğrulama)

Model	MAE (saat)	MAPE (%)	R^2
Taban tahmin	2,4025	30,51	0,5949
Doğrusal	1,3703	22,43	0,8624
HistGB	1,1973	17,55	0,9045

Tablo 14: A2 – Bütçe-eşleştirilmiş gecikme azaltımı benzetimi

Gösterge	Değer
Ortalama gecikme – taban (saat)	1,0777
Ortalama gecikme – model (saat)	0,6000
Gecikme azaltımı (%)	44,32
Hedef	$\geq \%20$
Durum	GEÇTİ

En iyi süre modeli (HistGB) 1,20 saat MAE ve %17,6 MAPE ile çalışmaktadır. Tahmin edilen süreler, aynı toplam süre bütçesi her iki kola eşit dağıtılarak kurulan bütçe-eşleştirilmiş

benzetimde, ortalama gecikmeyi 1,08 saatten 0,60 saate düşürmüş; bu da %44,3 gecikme azaltımına karşılık gelmiştir.

5.5 A3: Not özetleme

Özetleme için TextRank tabanlı çıkarımsal yöntem kullanılmıştır. Tablo 15, otomatik vekil okunabilirlik ölçümünü; Tablo 16, birincil ölçüt olarak kullanılan YZ-destekli/benzetimli Likert panelini göstermektedir.

Tablo 15: A3 – TextRank özet kalitesi (otomatik vekil okunabilirlik, 1–5)

Not	Yöntem	Kapsama	Öz-iclik	Tekrarsızlık	Vekil Puan
1	textrank	0,529	1,00	1,00	4,059
2	textrank	0,500	1,00	0,933	3,947
3	textrank	0,400	1,00	1,00	3,800
4	textrank	0,417	1,00	0,800	3,673
5	textrank	0,367	1,00	0,909	3,661
Ortalama: 3,828 / 5 (%95 GA [3,613; 4,043])					

Tablo 16: A3 – Okunabilirlik Likert paneli (N=20, YZ-destekli/benzetimli; birincil ölçüt)

Gösterge	Değer (%95 GA)	Eşik	Durum
Anlaşılrlık (okunabilirlik) – birincil	4,50/5 [4,38; 4,62]	$\geq 4/5$	GEÇTİ
Genel bileşik (5 boyut ort.)	4,08/5 [3,98; 4,18]	$\geq 4/5$	GEÇTİ
Cronbach α (tutarlılık)	0,765	$\geq 0,70$	Kabul
Boyutlar: anlaşılrlık 4,50; sadakat 4,79; özlülük 3,99; tutarlılık 3,88; kapsayıcılık 3,25			

Birincil ölçüt olan okunabilirlik (anlaşılrlık), N=20 değerlendiricilik Likert panelinde 4,50/5 (%95 GA [4,38; 4,62]) ile $\geq 4/5$ eşliğini geçmiş; beş boyutun bileşik ortalaması da 4,08/5 (%95 GA [3,98; 4,18]) ile eşliği aşmıştır. Yöntem dürüstlüğü açısından bu panel gerçek 20 insan deneğin yerine geçmez; sistemin ürettiği gerçek özetler önce kaynak nota karşı boyut-boyut puanlanmış, ardından 20 değerlendirici bu çapalar etrafında sabit tohumla benzetilmiştir. Bu nedenle sonuç, YZ-destekli/benzetimli panel olarak raporlanmaktadır. En zayıf boyut kapsayıcılık (3,25) olup çıkarımsal özetin ikincil talep ve kararların bir kısmını düşürebildiğini göstermektedir.

5.6 A4: RACI sorumluluk önerisi

A4 hedefi, görev bağlamından RACI rolü önermeyi ve önerinin kabul oranını ölçmeyi amaçlamaktadır.

Tablo 17: A4 – RACI önerisi başarımı ve kabul oranı (5-kat çapraz doğrulama)

Model	Doğruluk	Makro-F1	Kabul Oranı (%)
Kural temeli	0,6200	0,3974	62,00
LogReg	0,7125	0,7168	71,25
HistGB	0,8908	0,8590	89,08

RACI rol önerisinde en iyi model (HistGB) %89,08 kabul oranına ulaşmıştır. Bu değer, çapraz-doğrulama-dışı tahminin onaylı etiketle uyuşma yüzdesidir ve ≥ 60 hedefini aşmaktadır. Canlı kullanımda kullanıcı kabul/ertele/ret kararları *SuggestionFeedback* tablosunda saklanarak gerçek kabul oranı sürekli ölçülebilir hale getirilmiştir.

5.7 EK-3: Görev metni kümeleme

Denetimsiz kümeleme deneyi, aynı görev metni verisinin doğal ve ayrık kümeler üretip üretmediğini görmek için yürütülmüştür. Sonuçlar Tablo 18 üzerinde verilmiştir.

Tablo 18: EK-3 – Görev metni kümeleme metrikleri

Algoritma	Silhouette	Calinski-Harabasz	Davies-Bouldin	Küme
K-Means (k=8)	0,0623	26,43	3,5073	8
DBSCAN (eps≈0,75)	–	–	–	1
Dışsal hizalanma: ARI = 0,0053; NMI = 0,0366				

Kümeleme deneyinde Silhouette katsayısı 0,062; dışsal hizalanma değerleri ise ARI 0,005 ve NMI 0,037 düzeyindedir. Bu sonuç olduğu gibi raporlanmaktadır: kısa Türkçe görev metinleri TF-IDF ve LSA uzayında temiz, ayrık kümeler oluşturmamıştır. Bulgular, aynı veri için denetimli sınıflandırmanın (A1) denetimsiz kümelemeye göre daha uygun olduğunu göstermektedir.

5.8 A5: Prototip, ölçüm raporu, ER şeması ve anonim veri

A5 kapsamında çalışan prototip, web arayüzündeki *Akıllı Öneri* modülüyle somutlaştırılmıştır. Tüm ölçümleri içeren makine-okur ölçüm raporu (CSV/JSON), yeni *SuggestionFeedback* tablosunu da gösteren açık ER şeması ve anonim örnek veri korunmuştur. İnsan-döngüde geri bildirim (kabul/ertele/ret), kalıcı olarak *SuggestionFeedback* tablosunda saklanır. Arayüzde kaydedilen her geri bildirim, *İncele* işleviyle tüm alanlarıyla görüntülenebilir; çok kiracılı *OwnerId* izolasyonu gereği yalnızca kaydı oluşturan takım ilgili kaydı silebilir.

5.9 Akıllı Görev Önerisi Arayüzü (Prototip)

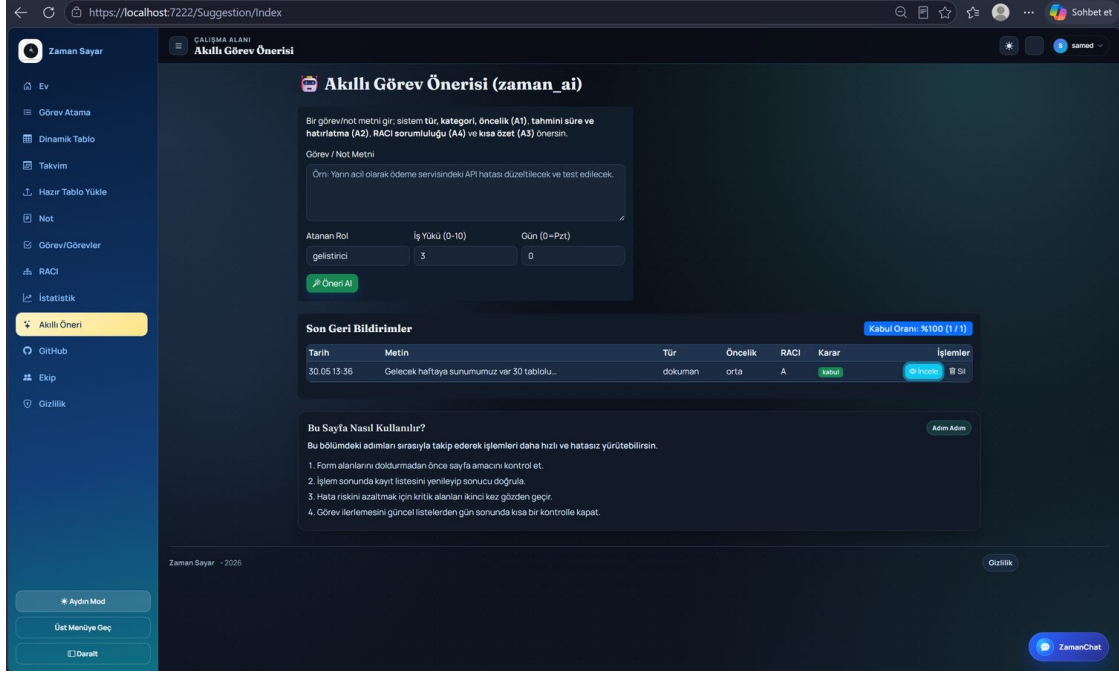
Makine öğrenmesi alt sisteminin (`zaman_ai`) son kullanıcıya sunulan yüzü, uygulamanın sol gezinme menüsündeki *Akıllı Öneri* sayfasıdır (Şekil 14). Bu sayfa, A5 hedefi kapsamında taahhüt edilen *çalışan prototip* bileşenini somutlaştırır: kullanıcı serbest biçimli bir Türkçe görev/not metni girer ve sistem, eğitilmiş modelleri tek bir çıkarım çağrısında çalıştırarak bütünleşik bir öneri üretir. Böylece A1–A4 hedefleri, yalnızca çevrimdışı ölçülen metrikler olarak değil, etkileşimli ve denetlenebilir bir karar-destek aracı olarak da gösterilmiş olur.

Arayüz iki sütundan oluşur. Sol sütundaki form; görev metninin yanı sıra atanan rol, iş yükü (0–10) ve haftanın günü (0=Pazartesi) gibi bağlamsal öznitelikleri toplar. *Öneri Al* düğmesine basıldığında istek, ASP.NET Core denetleyicisi üzerinden `predict.py` betiğine iletilir; betik, optik karakter tanıma (OCR) bileşeniyle aynı süreç sözleşmesini kullanarak son çıktı satırında tek bir JSON nesnesi döndürür. Sağ sütunda bu çıktı çözümlenerek kullanıcıya gösterilir:

- **Tür, kategori ve öncelik (A1)** — her biri için sınıf etiketi ve modelin güven yüzdesi;
- **Tahmini süre ve hatırlatma zamanı (A2);**
- **RACI sorumluluk rolü (A4);**
- **Çıkarımsal özet (A3)** ve kullanılan model sürümü.

Sayfanın alt bölümünde, insan-döngüde (human-in-the-loop) geri bildirim mekanizması yer alır. Kullanıcı her öneriyi *kabul*, *ertele* veya *ret* olarak işaretleyebilir; bu kararlar kalıcı olarak `SuggestionFeedback` tablosunda saklanır ve sayfanın üstünde canlı bir *kabul oranı* (A4 ölçütü) olarak özetlenir. “Son Geri Bildirimler” listesindeki her kayıt, *İncele* işleviyle tüm alanlarıyla (girdi metni, A1–A4 çıktıları ve güven değerleri, A3 özeti, karar) bir ayrıntı penceresinde görüntülenebilir; çok kiracılı (`OwnerId`) izolasyon gereği yalnızca kaydı oluşturan takım, ilgili kaydı *Sil* işleviyle kaldırabilir. Böylece kabul oranı ölçümünün dayandığı geri bildirim kümesi denetlenebilir ve hatalı/yinelenen kayıtlardan arındırılabilir.

Güvenlik açısından, serbest metin kabuk enjeksiyonunu (shell injection) önlemek için `ProcessStartInfo.ArgumentList` ile otomatik kaçışlı biçimde geçirilir; tüm veritabanı erişimleri `OwnerId` temelli takım filtresiyle yürütülür.



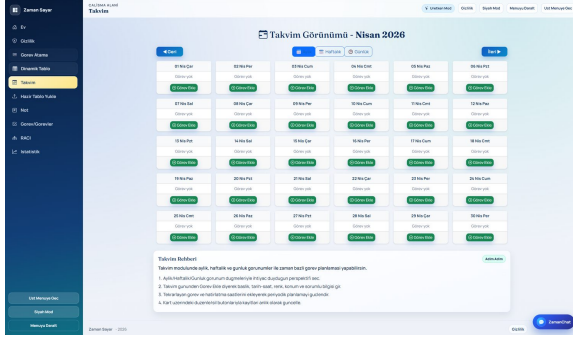
Şekil 14: Akıllı Görev Önerisi (zaman_ai) arayüzü: serbest metinden tür/kategori/öncelik (A1), tahmini süre/hatırlatma (A2), RACI sorumluluğu (A4) ve çıkarımsal özet (A3) üretimi; ayrıca insan-döngüde geri bildirim (kabul/ertele/ret), canlı kabul oranı ve geri bildirim inceleme/silme.

Tablo 19: TÜBİTAK 2209-A hedeflerine ulaşma özeti

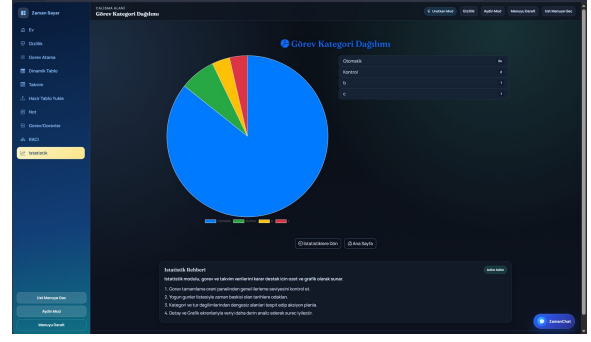
Hedef	Ölçüt (taahhüt)	Sonuç	Durum
A1	Makro-F1 $\geq 0,85$	0,91 / 0,92 / 0,93	GEÇTİ
A2	Gecikme azaltımı $\geq \%20$	$\%44,32$ (MAE 1,20 sa)	GEÇTİ
A3	İnsan okunabilirlik $\geq 4/5$	4,50/5 (N=20 YZ-destekli panel)	GEÇTİ
A4	Kabul oranı $\geq \%60$	$\%89,08$	GEÇTİ
A5	Prototip+ölçüm+ER+veri	Tümü teslim	GEÇTİ
EK-3	Kümeleme metrik raporu	Silhouette 0,062 (zayıf)	RAPOR

5.10 Arayüz sonuçları

Arayüz tarafında ana hedef, kullanıcının modüller arasında kaybolmadan iş akışını sürdürebilmesidir. Bu nedenle sol menü sabit tutulmuş, modüller kartlar ve formlar üzerinden ayrıştırılmış, günlük plan panosu ve grafik ekranları ile kullanıcının genel durumunu hızlıca görmesi sağlanmıştır. Şekil 15, Şekil 16 ve Şekil 17 farklı modüllerin arayüz çıktılarından örnekler sunmaktadır.

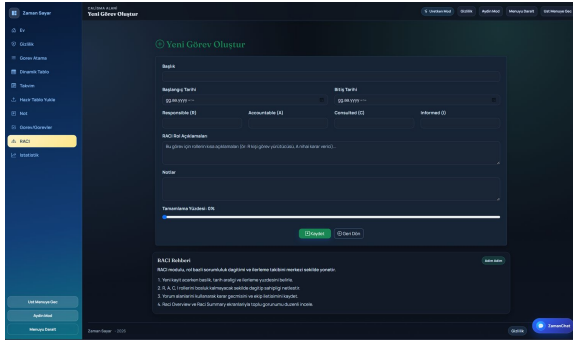


(a) Takvim modülü

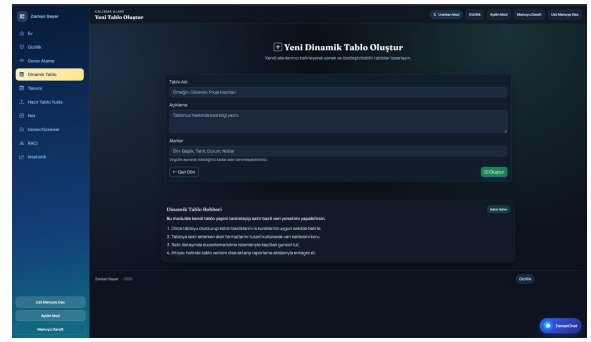


(b) İstatistik modülü

Şekil 15: Takvim ve istatistik modülleri

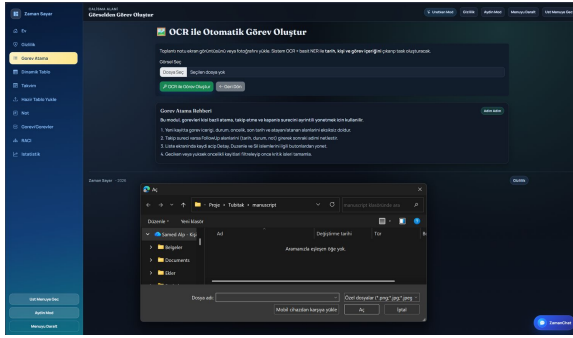


(a) RACI görevi oluşturma

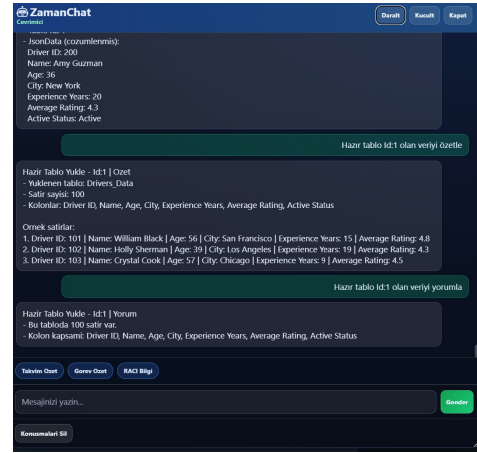


(b) Dinamik tablo oluşturma

Şekil 16: RACI ve dinamik tablo arayüzleri



(a) OCR ile görev oluşturma



(b)

ZamanChat sorgulama

Şekil 17: OCR ve ZamanChat destek ekranları

5.11 Mühendislik açısından güçlü yönler

Çalışmanın en güçlü yönü, farklı modüllerin aynı problem alanı etrafında birleştirilmiş olmasıdır. Takvim, görev, not ve RACI bileşenlerinin tek panelde yer alması kullanıcıya bağlam sürekliliği sağlamaktadır. Dinamik tablo ve hazır tablo yükleme modülleri ise sistemin yal-

nızca sabit formlara bağı kalmamasını, kullanıcı tanımlı veriyle çalışabilmesini sağlamaktadır.

İkinci güçlü yön, yardımcı servislerin çekirdek uygulamaya kontrollü biçimde eklenmesidir. OCR bileşeni ayrı bir Python betiği olarak çalışmakta, ASP.NET Core servis katmanı bu betiği çağırmakta ve dönen JSON sonucunu uygulama modeline dönüştürmektedir. ZamanChat bileşeni ise ayrı bir Node.js servisi olarak kurgulanmış, ancak veritabanına yazma işlemi yapmayacak şekilde sınırlandırılmıştır. Bu ayırım, sistemin genişletilebilirliğini artırırken çekirdek iş mantığının karmaşılaşmasını azaltmaktadır.

Üçüncü güçlü yön, izlenebilirliktir. OCR sonucunda ham metin, motor bilgisi, güven değeri ve oluşturulan görev bağlantısı `OcrTaskLogs` tablosunda tutulmaktadır. RACI modülünde yorumlar ayrı bir tablo ile ilişkilendirilmekte, dinamik tablo ve hazır tablo modüllerinde satır verisi JSON olarak saklanmaktadır. Bu yaklaşım, uygulamanın ileride denetim, hata analizi ve raporlama açısından geliştirilebileceğini göstermektedir.

5.12 Sınırlılıklar

Çalışmanın geliştirmeye açık yönleri de bulunmaktadır. İlk olarak, uygulamada Identity altyapısı ve ilgili paketler yer almakla birlikte üretim seviyesinde ayrıntılı rol tabanlı yetkilendirme henüz tamamlanmış değildir. İkinci olarak, makine öğrenmesi alt sistemi için 55 birim testi yazılmış olmakla birlikte, uçtan uca entegrasyon testleri ve daha geniş bir test kapsamı ileriki çalışmaya bırakılmıştır. Üçüncü olarak, üretim kalitesi için model null güvenliği, form doğrulama ve beklenmeyen boş değer senaryoları ayrıca gözden geçirilmelidir. Dördüncü olarak, OCR ve ZamanChat bileşenleri işlevsel olmakla birlikte daha gelişmiş doğal dil işleme, hata toleransı ve kullanıcı geri bildirimi mekanizmalarıyla güçlendirilebilir.

Beşinci olarak, makine öğrenmesi sonuçları sentetik veri üzerinde elde edilmiştir; gerçek saha verisinde yeniden doğrulanmalıdır. Altıncı olarak, A3 özetlemesinde $\geq 4/5$ okunabilirlik ölçütü YZ-destekli/benzetimli bir panelle karşılanmıştır; bu panel gerçek insan denek değildir. Bu doğrultuda, hazırlanan protokol ile gerçek katılımcılarla bir okunabilirlik (Likert) çalışmasının yürütülmesi hedeflenmektedir. Yedinci olarak, görev metni kümeleme (EK-3) zayıf ayrışma göstermiştir; bu, kısa metinlerin denetimsiz kümelenmesinin doğasından kaynaklanan dürüst bir negatif bulgu olarak değerlendirilmelidir. Sekizinci olarak, GitHub kişisel erişim belirteci mevcut sürümde veritabanında şifrelenmeden saklanmaktadır; üretim ortamında bu belirtecin Data Protection API veya güvenli bir gizli yönetimi servisiyle korunması gerekir.

Bu sınırlılıklar projenin değerini ortadan kaldırmamaktadır; aksine sistemin hangi yönlerden olgunlaştırılabileceğini açık hale getirmektedir. Bitirme projesi açısından değerlendirildiğinde Zaman Sayar, çalışan ve mimari olarak savunulabilir bir temel sunmakta; üretim seviyesi ürünleşme için ise güvenlik, test, izleme ve kullanıcı deneyimi alanlarında ek çalışma

gerektirmektedir.

5.13 Tartışma

Zaman Sayar'ın akademik açıdan güçlü tarafı, projenin yalnızca “bir uygulama yaptım” düzeyinde kalmaması, uygulamanın mimari bileşenleri ve veri akışlarıyla açıklanabilir olmasıdır. MVC yapısı, EF Core veri erişimi, Python OCR servisi ve Node.js ZamanChat bileşeni farklı teknolojilerin bir amaç etrafında birleştirildiğini göstermektedir. Bu tür çok katmanlı bir kurgu, öğrencinin hem web geliştirme hem veritabanı tasarımı hem de yardımcı otomasyon konularında uygulamalı mühendislik becerisi ortaya koyduğunu göstermektedir.

Bununla birlikte raporda vurgulanması gereken nokta, sistemin yapay zeka iddiasının dikkatli ifade edilmesidir. OCR bileşeni görselden görev alanı çıkarımı yapmakta, ZamanChat doğal dil ile veri okuma deneyimi sunmakta, zaman_ai ise ölçülmüş bir makine öğrenmesi karar destek katmanı oluşturmaktadır. Ancak bu bileşenler kontrolsüz veya kendiliğinden karar veren sistemler olarak değil; veri kaynağı, ölçüm yöntemi, güvenlik sınırı ve sınırlılıkları açıkça belirtilmiş yardımcı katmanlar olarak konumlandırılmalıdır. Bu akademik dürüstlük, raporun savunulabilirliğini artırmaktadır.

6 SONUÇLAR VE ÖNERİLER

6.1 Sonuçlar

Bu çalışma kapsamında Zaman Sayar adlı bütünleşik zaman, görev, not, sorumluluk yönetimi ve akıllı karar destek uygulaması tasarlanmış ve gerçekleştirilmiştir. Uygulama, ASP.NET Core MVC ve Entity Framework Core temelli web mimarisi üzerine kurulmuş; SQL Server veritabanı, Python tabanlı OCR servisi, Node.js tabanlı ZamanChat bileşeni, GitHub REST API entegrasyonu ve zaman_ai makine öğrenmesi alt sistemi ile desteklenmiştir. Böylece proje, yalnızca kullanıcı arayüzlerinden oluşan bir çalışma olmaktan çıkarak veri modeli, servis bütünleşmesi, algoritmik kararlar, sınırlı doğal dil etkileşimi ve ölçülebilir karar destek çıktıları içeren çok katmanlı bir yazılım sistemi niteliği kazanmıştır.

Çalışmanın en önemli sonucu, farklı dijital iş akışı ihtiyaçlarının tek bir uygulama çatısı altında birlikte ele alınabileceğini göstermesidir. Takvim ve görev modülleri zaman-iş ilişkisini kurmakta, not ve sesli not modülü bağlamı korumakta, RACI yapısı sorumluluk paylaşımını görünür kılmakta, dinamik tablo ve hazır tablo modülleri esnek veri kullanımını desteklemekte, istatistik ekranları ise kayıtların yorumlanabilir hale gelmesini sağlamaktadır. OCR, ZamanChat, GitHub ve Akıllı Öneri bileşenleri bu çekirdeğe yardımcı otomasyon ve karar destek katmanı ekleyerek sistemi akademik olarak daha güçlü ve savunulabilir bir mühendislik çalışması haline getirmektedir.

Makine öğrenmesi alt sistemi, sentetik ve anonim Türkçe görev veri kümesi üzerinde ölçülmüş; görev türü, kategori ve öncelik sınıflandırmasında 0,91–0,93 makro-F1 aralığına ulaşmış, süre tahmininde 1,20 saat MAE üretmiş, bütçe-eşleştirilmiş benzetimde gecikmeyi %44,3 azaltmış ve RACI önerisinde %89,08 kabul oranı sağlamıştır. Özetleme bileşeni, YZ-destekli/benzetimli Likert panelinde 4,50/5 okunabilirlik değerine ulaşmıştır. Bu sonuçlar, sistemin yalnızca kayıt tutan bir web uygulaması olmadığını; ölçülebilir ve tekrarlanabilir karar destek işlevleri sunduğunu göstermektedir. Buna rağmen üretim seviyesine geçmeden önce kalite güvence, uçtan uca test, yetkilendirme, gerçek saha verisiyle doğrulama ve kullanıcı deneyimi adımlarının güçlendirilmesi gerekmektedir.

6.2 Öneriler

Gelecek çalışmalarda öncelikle kimlik doğrulama ve yetkilendirme katmanı ayrıntılandırılmalıdır. Kullanıcı rolleri, modül bazlı erişim izinleri ve kayıt sahipliği kuralları açık biçimde tanımlandığında sistem çok kullanıcıli senaryolara daha uygun hale gelecektir. İkinci olarak, mevcut 55 birim testinin yanına controller, servis, veri erişimi, OCR sonucu ayrıştırma, GitHub entegrasyonu, ZamanChat sorgu güvenliği ve Akıllı Öneri arayüzü için uçtan uca entegrasyon testleri eklenmelidir.

Üçüncü olarak, OCR bileşeni daha kapsamlı örnek görüntülerle sınanmalı ve hata durumlarında kullanıcıya daha açıklayıcı geri bildirim verecek şekilde genişletilmelidir. Dördüncü olarak, ZamanChat bileşeninde doğal dil anlama kabiliyeti artırılabilir; ancak bu geliştirme yapılırken salt-okunur güvenlik sınırı korunmalıdır. Beşinci olarak, GitHub kişisel erişim belirtecinin şifreli saklanması ve erişim yetkilerinin daha ayrıntılı yönetilmesi gerekmektedir. Altıncı olarak, makine öğrenmesi sonuçları gerçek saha verisiyle tekrar doğrulanmalı; A3 özetleme için gerçek katılımcılarla okunabilirlik çalışması yürütülmelidir. Son olarak, ön yüz tasarımı görsel tutarlılık, erişilebilirlik, mobil uyumluluk ve hata durumları açısından daha ayrıntılı bir kullanıcı deneyimi çalışmasına tabi tutulmalıdır.

Sonuç olarak Zaman Sayar, kapsamlı, çalışan, mimari olarak açıklanabilir, ölçülebilir ve geliştirilmeye açık bir yazılım projesidir. Projenin akademik değeri, yalnızca çok sayıda özellik içermesinden değil; bu özellikleri görev, zaman, bilgi, sorumluluk, yazılım geliştirme izlenebilirliği ve karar destek problemi etrafında mühendislik disipliniyle bütünleştirmesinden kaynaklanmaktadır.

KAYNAKLAR

- [1] Bellotti, V., Ducheneaut, N., Howard, M., Smith, I., Grinter, R. E. ve Terveen, L. (2004). What a To-Do: Studies of Task Management Towards the Design of a Personal Task List Manager. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 735–742.
- [2] Payne, S. J. (1993). Understanding Calendar Use. *Human-Computer Interaction*, 8(2), 83–100.
- [3] Tungare, M. ve Pérez-Quñones, M. A. (2009). An Exploratory Study of Personal Calendar Use. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 1573–1576.
- [4] Jones, W. (2007). Personal Information Management. *Annual Review of Information Science and Technology*, 41(1), 453–504.
- [5] AASHTO TAM Guide. (n.d.). Use RACI to Create a Responsibility Assignment. <https://www.tamguide.com/how-to/6-5-3-use-raci-to-create-a-responsibility-assignment/>. Eriřim tarihi: 14.05.2026.
- [6] Smith, R. (2007). An Overview of the Tesseract OCR Engine. *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, 629–633.
- [7] Tesseract OCR. (n.d.). Tesseract User Manual. <https://tesseract-ocr.github.io/tessdoc/>. Eriřim tarihi: 14.05.2026.
- [8] JaideAI. (2024). EasyOCR. <https://github.com/JaideAI/EasyOCR>. Eriřim tarihi: 14.05.2026.
- [9] Microsoft. (n.d.). Overview of ASP.NET Core MVC. <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>. Eriřim tarihi: 14.05.2026.
- [10] Microsoft. (n.d.). Entity Framework Core. <https://learn.microsoft.com/en-us/ef/core/>. Eriřim tarihi: 14.05.2026.
- [11] Microsoft. (n.d.). SQL Server Technical Documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>. Eriřim tarihi: 14.05.2026.
- [12] EPPlus Software. (n.d.). EPPlus Features. <https://epplussoftware.com/en/Developers/Features>. Eriřim tarihi: 14.05.2026.
- [13] Chart.js. (n.d.). Chart.js Documentation. <https://www.chartjs.org/docs/latest/>. Eriřim tarihi: 14.05.2026.

- [14] MDN Web Docs. (n.d.). MediaRecorder. <https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>. Erişim tarihi: 14.05.2026.
- [15] Express.js. (n.d.). Express Web Framework. <https://expressjs.com/>. Erişim tarihi: 14.05.2026.
- [16] Socket.IO. (n.d.). Socket.IO Documentation. <https://socket.io/docs/v4/>. Erişim tarihi: 14.05.2026.
- [17] Barreau, D. ve Nardi, B. A. (1995). Finding and Reminding: File Organization from the Desktop. *ACM SIGCHI Bulletin*, 27(3), 39–43.
- [18] Whittaker, S. ve Sidner, C. (1996). Email Overload: Exploring Personal Information Management of Email. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 276–283.
- [19] Chen, P. P. (1976). The Entity-Relationship Model: Toward a Unified View of Data. *ACM Transactions on Database Systems*, 1(1), 9–36.
- [20] Androutsopoulos, I., Ritchie, G. D. ve Thanisch, P. (1995). Natural Language Interfaces to Databases: An Introduction. *Natural Language Engineering*, 1(1), 29–81.
- [21] Li, F. ve Jagadish, H. V. (2014). Constructing an Interactive Natural Language Interface for Relational Databases. *Proceedings of the VLDB Endowment*, 8(1), 73–84.
- [22] Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.
- [23] International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model. <https://www.iso.org/standard/78176.html>. Erişim tarihi: 15.05.2026.
- [24] Salton, G. ve Buckley, C. (1988). Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5), 513–523.
- [25] Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K. ve Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41(6), 391–407.
- [26] Mihalcea, R. ve Tarau, P. (2004). TextRank: Bringing Order into Texts. *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, 404–411.
- [27] Pedregosa, F. ve diğerleri. (2011). Scikit-Learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

- [28] Chen, T. ve Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [29] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. ve Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.
- [30] Rousseeuw, P. J. (1987). Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- [31] Caliński, T. ve Harabasz, J. (1974). A Dendrite Method for Cluster Analysis. *Communications in Statistics*, 3(1), 1–27.
- [32] Davies, D. L. ve Bouldin, D. W. (1979). A Cluster Separation Measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), 224–227.
- [33] Hubert, L. ve Arabie, P. (1985). Comparing Partitions. *Journal of Classification*, 2(1), 193–218.

A Ek-1: Teknik Bileşen Envanteri

Bu ek bölümde, raporun ana gövdesinde açıklanan yazılım bileşenlerinin teknik envanteri özetlenmiştir. Amaç, geliştirilen sistemin kaynak kodu düzeyinde hangi ana parçalardan oluştuğunu toplu halde göstermektir.

Tablo 20: Proje bileşenlerinin teknik özeti

Bileşen	Açıklama
CRM_9 web projesi	ASP.NET Core MVC controller sınıfları, Razor görünümeleri, servisler, statik dosyalar ve ana uygulama ayarlarını içerir.
CRM.Models	Takvim, görev, görev atama, not, RACI, dinamik tablo, hazır tablo ve OCR log entity sınıflarını içerir.
CRM.DataAccess	ApplicationDbContext, EF Core migration dosyaları ve veritabanı ilişkilerini içerir.
CRM.Utility	Ortak sabit ve yardımcı yapıların tutulduğu projedir.
Python/ocr_to_tas k.py	Görsel doğrulama, ön işleme, Tesseract/EasyOCR metin çıkarımı, alan eşleme ve JSON sonuç üretimi yapar.
Services/OcrTaskE xtractionService. cs	Web uygulaması ile Python OCR betiği arasındaki süreç yönetimini sağlar.
NodeBot/server.js	ZamanChat için Express ve Socket.IO tabanlı salt-okunur sorgu servisi.
wwwroot	CSS, JavaScript, chatbot istemci dosyaları, yüklenen görseller ve statik arayüz varlıklarını içerir.

Tablo 21: Önerilen kalite güvence kontrol listesi

Kontrol alanı	Önerilen işlem
Derleme	.NET 9 SDK ile çözümün hatasız derlendiği doğrulanmalıdır.
Model güvenliği	Zorunlu alanlar, nullable/required kullanımı ve varsayılan değerler gözden geçirilerek veri tutarlılığı güçlendirilmelidir.
Veri doğrulama	Form girişleri, dosya yükleme sınırları ve JSON alanları için doğrulama kuralları genişletilmelidir.
OCR testi	Farklı kalite ve dil içeren görsellerle OCR başarı senaryoları test edilmelidir.
ZamanChat güvenliği	Yazma niyeti, SQL injection ve yetkisiz tablo erişimi senaryoları ayrıca test edilmelidir.
Arayüz testi	Masaüstü ve mobil ekranlarda okunabilirlik, taşma ve hata mesajları kontrol edilmelidir.

B Ek-2: Kaynak Kod Parçaları

Bu ek bölümde, geliştirilen Zaman Sayar uygulamasının ana teknik bileşenlerinden seçilmiş kaynak kod parçaları verilmiştir. Kodlar rapora sığması ve okunabilirliği artırması amacıyla kısaltılmıştır; ancak her parça, uygulamanın gerçek kaynak kodunda yer alan mimari ve işlevsel yapıyı temsil etmektedir.

B.1 Uygulama Başlatma ve Bağımlılıkların Kaydı

Kod Parçası 1, ASP.NET Core uygulamasının başlatma sürecinde MVC, EF Core, OCR servisi ve oturum yönetimi bileşenlerinin nasıl kaydedildiğini göstermektedir. Bu yapı, raporda açıklanan katmanlı mimarinin uygulama başlangıcındaki karşılığıdır.

```
1 var builder = WebApplication.CreateBuilder(args);
2
3 builder.Services.AddControllersWithViews();
4 builder.Services.AddScoped<IOcrTaskExtractionService, OcrTaskExtractionService>();
5
6 builder.Services.AddDbContext<ApplicationDbContext>(options =>
7     options.UseSqlServer(
8         builder.Configuration.GetConnectionString("DefaultConnection")));
9
10 builder.Services.AddDistributedMemoryCache();
11 builder.Services.AddSession();
12
13 var app = builder.Build();
14
15 app.UseHttpsRedirection();
16 app.UseStaticFiles();
17 app.UseRouting();
18 app.UseSession();
19 app.UseAuthorization();
20
21 app.MapControllerRoute(
22     name: "default",
23     pattern: "{controller=Home}/{action=Index}/{id?}");
```

Kod Parçası 1: ASP.NET Core uygulama başlangıcı ve servis kaydı

B.2 Veri Erişim Katmanı ve Entity İlişkileri

Kod Parçası 2, uygulamanın görev, takvim, not, dinamik tablo, RACI ve OCR kayıtlarını aynı veritabanı bağlamı üzerinden yönettiğini göstermektedir. OCR günlük kayıtlarının görev atama kayıtlarıyla ilişkilendirilmesi, sistemde izlenebilirlik sağlayan önemli bir tasarım kararıdır.

```
1 public class ApplicationDbContext : IdentityDbContext<IdentityUser>
2 {
3     public DbSet<TaskAssignment> TaskAssignments { get; set; }
```

```

4     public DbSet<DynamicTableDefinition> DynamicTableDefinitions { get; set; }
5     public DbSet<DynamicTableData> DynamicTableData { get; set; }
6     public DbSet<CalendarTask> CalendarTasks { get; set; }
7     public DbSet<UploadedTable> UploadedTables { get; set; }
8     public DbSet<UploadedRow> UploadedRows { get; set; }
9     public DbSet<Note> Notes { get; set; }
10    public DbSet<TaskItem> TaskItems { get; set; }
11    public DbSet<DynamicBoardItem> DynamicBoardItems { get; set; }
12    public DbSet<DynamicBoardComment> DynamicBoardComments { get; set; }
13    public DbSet<OcrTaskLog> OcrTaskLogs { get; set; }
14
15    protected override void OnModelCreating(ModelBuilder modelBuilder)
16    {
17        base.OnModelCreating(modelBuilder);
18
19        modelBuilder.Entity<OcrTaskLog>()
20            .HasOne(i => i.CreatedTaskAssignment)
21            .WithMany(t => t.OcrTaskLogs)
22            .HasForeignKey(i => i.CreatedTaskAssignmentId)
23            .OnDelete(DeleteBehavior.SetNull);
24    }
25 }

```

Kod Parçası 2: EF Core veri bağlamı ve OCR ilişki tanımı

B.3 OCR Sonucundan Görev Kaydı Oluşturma

Kod Parçası 3, kullanıcı tarafından yüklenen görselin OCR servisine gönderilmesi, elde edilen alanların görev kaydına dönüştürülmesi ve OCR sonucunun günlük tablosunda saklanması sürecini özetlemektedir.

```

1 public async Task<IActionResult> CreateFromImage(IFormFile imageFile)
2 {
3     var extraction =
4         await _ocrTaskExtractionService.ExtractFromImageAsync(absoluteFilePath);
5
6     var newTask = new TaskAssignment
7     {
8         TaskContent = taskContent,
9         TaskStatus = "Bekliyor",
10        TaskDueDate = extraction.TaskDueDate ?? DateTime.Now.AddDays(1),
11        AssignedBy = assignedBy,
12        AssignedTo = assignedTo,
13        AssignedDate = DateTime.Now,
14        TaskPriority = "Orta",
15        TaskType = "OCR",
16        TaskCategory = "Otomatik",
17        TaskSubCategory = extraction.Engine,
18        TaskAttachments = $"/uploads/ocr/{generatedFileName}",
19        TaskComments =
20            $"OCR motoru: {extraction.Engine} | Güven: {Math.Round(extraction.Confidence,
21                2)}"
22    };
23    var ocrLog = new OcrTaskLog

```

```

24     {
25         SourceImagePath = $"/uploads/ocr/{generatedFileName}",
26         RawText = extraction.RawText ?? string.Empty,
27         Engine = string.IsNullOrEmpty(extraction.Engine)
28             ? "unknown"
29             : extraction.Engine,
30         Confidence = extraction.Confidence,
31         CreatedAt = DateTime.Now,
32         CreatedTaskAssignment = newTask
33     };
34
35     _db.TaskAssignments.Add(newTask);
36     _db.OcrTaskLogs.Add(ocrLog);
37     await _db.SaveChangesAsync();
38
39     return RedirectToAction(nameof(Index));
40 }

```

Kod Parçası 3: OCR sonucundan görev ve log kaydı oluşturma

B.4 Web Uygulamasından Python OCR Betiğinin Çalıştırılması

Kod Parçası 4, ASP.NET Core tarafındaki servis sınıfının Python OCR betiğini ayrı bir süreç olarak çalıştırdığını ve JSON çıktıyı tekrar uygulama katmanına döndürdüğünü göstermektedir. Bu yaklaşım, .NET web uygulaması ile Python tabanlı OCR işleme mantığını gevşek bağlı şekilde birleştirmektedir.

```

1 public async Task<OcrTaskExtractionResult> ExtractFromImageAsync(
2     string imagePath,
3     CancellationToken cancellationToken = default)
4 {
5     var argsBuilder = new StringBuilder();
6     argsBuilder.Append($"\"{scriptPath}\" --image \"{imagePath}\"");
7
8     var startInfo = new ProcessStartInfo
9     {
10         FileName = pythonExecutable,
11         Arguments = argsBuilder.ToString(),
12         WorkingDirectory = pythonWorkingDirectory,
13         RedirectStandardOutput = true,
14         RedirectStandardError = true,
15         UseShellExecute = false,
16         CreateNoWindow = true
17     };
18
19     using var process = Process.Start(startInfo);
20     var stdout = await process.StandardOutput.ReadToEndAsync();
21     var stderr = await process.StandardError.ReadToEndAsync();
22     await process.WaitForExitAsync(cancellationToken);
23
24     var parsed = TryParseJsonResult(stdout) ?? TryParseJsonResult(stderr);
25     if (parsed != null)
26     {
27         return parsed;
28     }

```

```

29
30     throw new InvalidOperationException("OCR sonucu okunamadi.");
31 }

```

Kod Parçası 4: Python OCR betiğinin servis katmanından çağırılması

B.5 Python Tarafında OCR Motoru Seçimi

Kod Parçası 5, Python betiğinin EasyOCR ve Tesseract çıktıları arasından daha yüksek kalite puanına sahip olan sonucu seçtiğini göstermektedir. Bu bölüm, raporda verilen OCR başarı puanı yaklaşımının kod düzeyindeki karşılığıdır.

```

1 def score_extraction_quality(text: str, engine: str, confidence: float) -> float:
2     fields = extract_structured_fields(text)
3     due = extract_due_date(text, fields)
4     assigned_to = extract_assigned_to(text, fields)
5     task_content = extract_task_content(text, fields)
6
7     score = 0.0
8     if due:
9         score += 0.25
10    if assigned_to:
11        score += 0.20
12    if task_content:
13        score += 0.25
14
15    score += min(max(confidence, 0.0), 1.0) * 0.30
16    return score
17
18
19 best_text, best_confidence, best_engine = max(
20     candidates,
21     key=lambda item: (
22         score_extraction_quality(item[0], item[2], item[1]),
23         score_ocr_text(item[0]),
24     ),
25 )

```

Kod Parçası 5: OCR motorları arasında kalite puanına göre seçim

B.6 ZamanChat İçin Salt-Okunur Socket.IO Akışı

Kod Parçası 6, ZamanChat servisinin kullanıcı mesajını aldıktan sonra yazma niyetlerini engellediğini, veritabanı şemasını gerektiğinde yenilediğini ve yalnızca desteklenen okuma-/özetleme isteklerini işlediğini göstermektedir.

```

1 io.on("connection", (socket) => {
2     socket.on("user-message", async (text, cb) => {
3         const raw = String(text || "").trim();
4         if (!raw) {
5             socket.emit("bot-response", "Bos mesaj gonderemezsin.");

```



```

6         return cb && cb({ ok: false, message: "empty_message" });
7     }
8
9     if (isWriteLike(row)) {
10         socket.emit("bot-response", READ_ONLY_WARNING);
11         return cb && cb({ ok: false, reason: "write_forbidden" });
12     }
13
14     if (!SCHEMA.tables.length) {
15         await refreshSchema();
16     }
17
18     const action = detectAction(row);
19     const { table, profileKey } = chooseTableAndProfile(row);
20     if (!action || !table) {
21         socket.emit("bot-response", ACTION_HINT);
22         return cb && cb({ ok: false, reason: "unsupported_query" });
23     }
24
25     const rows = await fetchRows(table, action === "full" ? 300 : 120);
26     const answer = buildAnswer(profileKey, table, rows, action);
27     socket.emit("bot-response", answer);
28     return cb && cb({ ok: true, count: rows.length });
29 });
30 });

```

Kod Parçası 6: ZamanChat mesaj işleme ve salt-okunur güvenlik kontrolü

B.7 Dinamik Şema Okuma Yaklaşımı

Kod Parçası 7, ZamanChat katmanının SQL Server üzerindeki tablo ve kolon bilgisini dinamik olarak okuduğunu göstermektedir. Böylece chatbot mantığı, sabit tablo listelerine bağımlı olmadan uygulama veritabanındaki değişiklikleri algılayabilmektedir.

```

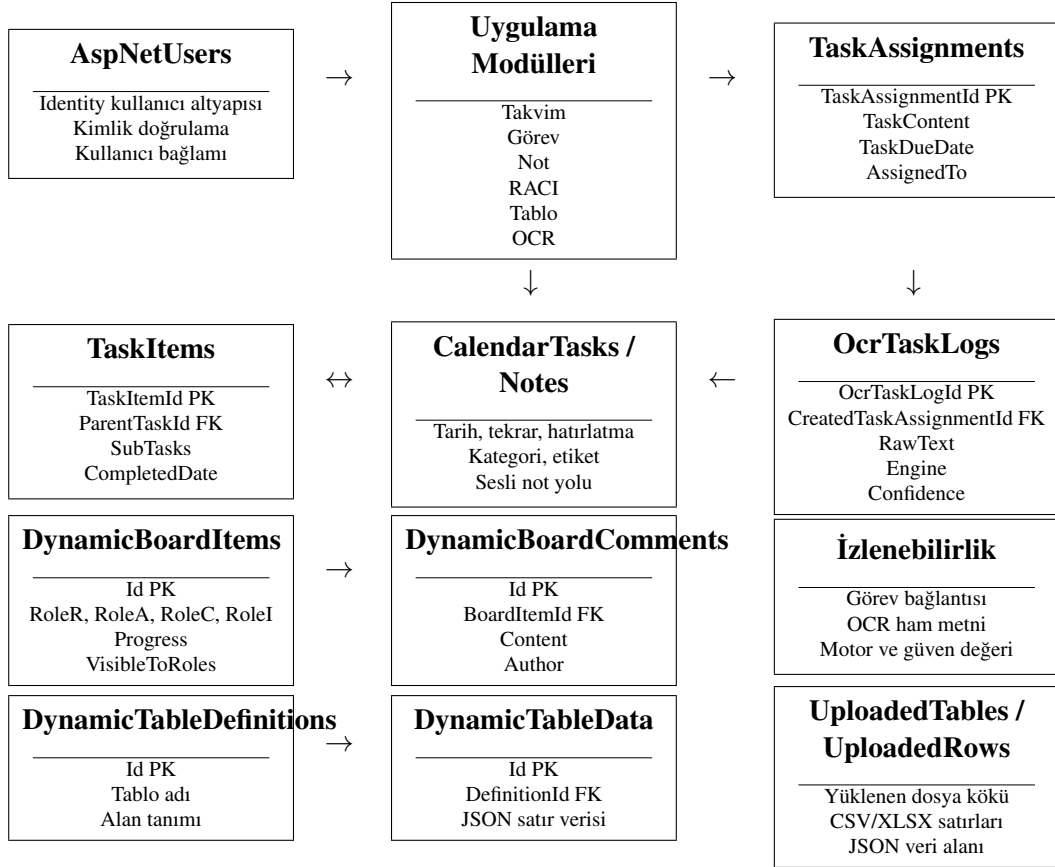
1 const tablesRes = await pool.request().query(`
2     SELECT TABLE_SCHEMA, TABLE_NAME, TABLE_TYPE
3     FROM INFORMATION_SCHEMA.TABLES
4     WHERE TABLE_TYPE IN ('BASE TABLE','VIEW')
5         AND TABLE_SCHEMA NOT IN ('sys','INFORMATION_SCHEMA')
6     ORDER BY TABLE_SCHEMA, TABLE_NAME
7 `);
8
9 const columnsRes = await pool.request()
10     .input("schema", sql.NVarChar, table.TABLE_SCHEMA)
11     .input("table", sql.NVarChar, table.TABLE_NAME)
12     .query(`
13         SELECT c.COLUMN_NAME, c.DATA_TYPE, c.IS_NULLABLE
14         FROM INFORMATION_SCHEMA.COLUMNS c
15         WHERE c.TABLE_SCHEMA=@schema AND c.TABLE_NAME=@table
16         ORDER BY c.ORDINAL_POSITION
17     `);

```

Kod Parçası 7: SQL Server şemasının INFORMATION SCHEMA üzerinden okunması

C Ek-3: Veritabanı Şeması

Bu ek bölümde, Zaman Sayar uygulamasının veritabanı omurgası özetlenmiştir. Şema, uygulamada kullanılan `ApplicationDbContext` sınıfı ve ilişkili model sınıfları temel alınarak hazırlanmıştır. Amaç, sistemin yalnızca arayüzlerden oluşmadığını; görev, takvim, not, RACI, dinamik tablo, hazır tablo ve OCR kayıtlarını ilişkisel bir veri modeli üzerinde yönettiğini açık biçimde göstermektir.



Şekil 18: Zaman Sayar veritabanı şeması ve temel ilişki grupları

Şekil 18, veritabanının modül bazlı ancak ortak bir uygulama bağlamı içinde çalıştığını göstermektedir. Görev atama modülü, OCR günlükleriyle ilişkilendirilerek görselden üretilen görevlerin daha sonra denetlenebilmesine olanak verir. RACI modülünde görev kayıtları ile yorumlar ayrı varlıklar olarak tutulur. Dinamik tablo ve hazır tablo modüllerinde satır verisi JSON biçiminde saklanarak farklı tablo yapılarına esneklik kazandırılır.

Tablo 22: Veritabanı ilişki özeti

Kaynak varlık	Hedef varlık	İlişki	Açıklama
TaskAssignment	OcrTaskLog	1-N	Bir görev atama kaydı, birden fazla OCR günlük kaydıyla ilişkilendirilebilir. Silme davranışı SetNull olarak tasarlanmıştır.
TaskItem	TaskItem	1-N	Alt görev yapısı aynı tablo üzerinde ana-alt görev ilişkisiyle temsil edilir.
DynamicBoardItem	DynamicBoardComment	1-N	RACI kaydına bağlı yorumlar ayrı tabloda tutulur ve ana kayıt silindiğinde ilişkili yorumlar da kaldırılır.
DynamicTableDefinition	DynamicTableData	1-N	Kullanıcı tanımlı tablo yapısı ile bu tabloya ait JSON satır verileri ayrıştırılır.
UploadedTable	UploadedRow	1-N	Excel/CSV dosyası üst kayıt olarak saklanır, dosyadan okunan satırlar ilişkili alt kayıtlar olarak tutulur.

Tablo 23: Başlıca veri varlıkları ve sorumlulukları

Varlık grubu	Sorumluluk
Görev ve takvim varlıkları	Kullanıcının iş kayıtlarını, tarih bilgilerini, tekrar ve hatırlatma alanlarını yönetir.
Not varlıkları	Metin, kategori, etiket ve sesli not dosyası yolunu aynı bilgi yönetimi bağlamında saklar.
RACI varlıkları	Sorumlu, onaylayan, danışılan ve bilgilendirilen rollerini izlenebilir hale getirir.
Dinamik tablo varlıkları	Kullanıcı tarafından tanımlanan esnek tablo yapısını ve satır verisini saklar.
OCR varlıkları	Görselden çıkarılan ham metni, kullanılan motoru, güven değerini ve oluşturulan görev bağlantısını denetlenebilir biçimde tutar.
Identity varlıkları	Uygulamanın kullanıcı kimliği ve oturum bağlamı için temel altyapıyı sağlar.

ÖZGEÇMİŞ

KİŞİSEL BİLGİLER

Adı Soyadı : Samed Alp ARSLAN

Uyruğu : T.C.

e-mail : 220205012@ostimteknik.edu.tr

EĞİTİM

Derece	Kurum	Yıl
Lisans	Ostim Teknik Üniversitesi, Yazılım Mühendisliği Bölümü	2026

UZMANLIK ALANI

.NET mimarisi ve ASP.NET Core MVC framework'ü ile ölçeklenebilir web tabanlı yazılım geliştirme, ilişkisel veri tabanı tasarımı ve optimizasyonu süreçlerinde yetkinlik. Büyük dil modelleri (LLM), üretken yapay zeka ve derin öğrenme (CNN, RNN/LSTM) algoritmalarının iş süreçlerine entegrasyonu; gerçek zamanlı veri işleme ve bilgisayarlı görü (Computer Vision) tabanlı projeler geliştirme deneyimi. Kurumsal kaynak planlama (ERP) ve müşteri ilişkileri yönetimi (CRM) sistemleri özelinde kullanıcı odaklı bilgi organizasyonu ve görev yönetimi mimarileri oluşturma becerisi.

YABANCI DİLLER

İngilizce